6장
여러 데이터를 한 번에 묶어 표현하기: 리스트

지금까지 프로그래밍의 기본이 되는 입력과 출력, 변수, 조건문과 반복문을 살펴보았습니다. 수학의 사칙연산과 할머니 도형식을 배운 것과 같습니다.

이제부터는 파이썬으로 조금 더 재미있고 복잡한 작업을 하는 데 필요한 개념을 배워 보겠습니다. 먼저 이 장에서는 리스트를 배웁니다. 리스트만 알아도 데이터 분석 프로젝트를 할 수 있으니 아 강해서 배운 내용을 활용해 자기 것으로 만들 수 있습니다.

6.1

여러 데이터 한 번에 묶기

6.1.1 리스트란

여러분이 월별 지출을 정리하는 가계부 프로그램을 만든다고 가정해 봅시다. 지출을 월별로 저장해야 하므로 지금까지 배운 대로 한다면 다음처럼 12개의 변수가 필요하겠죠.

```
jan = 25
feb = 36
...
nov = 40
dec = 25
```

변수가 12개라서 적어 보일 수 있지만, 매일 지출을 정리한다면 어떨까요? 365개의 변수를 만들어야 합니다. 코드로 작성하기가 만만치 않습니다. 어떻게 해야 많은 데이터를 좀 더 효율적으로 다룰 수 있을까요?

데이터를 다룰 때 많이 사용하는 방법으로 표가 있습니다. 월별 지출을 표로 만들어 봅시다.

▼ 표 6-1 월별 지출표

월	1월	2월	3월	4월	5월	6월	7월	8월	9월	10월	11월	12월
지출	25	36	8	53	24	56	38	79	94	20	40	25

헤더를 제외하고 총 12칸짜리 표를 만들 수 있습니다. 표에서 지출은 월을 기준으로 순서대로 나열됩니다. 첫 번째 칸은 1월 지출, 두 번째 칸은 2월 지출, 열두 번째 칸은 12월 지출입니다. 12월 지출이 궁금하면 '지출표의 열두 번째 칸' 또는 '지출표의 마지막 칸'으로 찾을 수 있습니다. 표로 구조화하니 데이터를 보기가 한결 편합니다.

파이썬에서도 이런 표 형태로 데이터를 저장할 수 있는데, 이를 리스트(list)라고 합니다. 리스트는 여러 개의 데이터를 묶어서 변수 하나에 담아 표현하는 파이썬의 데이터 타입입니다. 리스트를 만드는 방법은 다음과 같습니다.

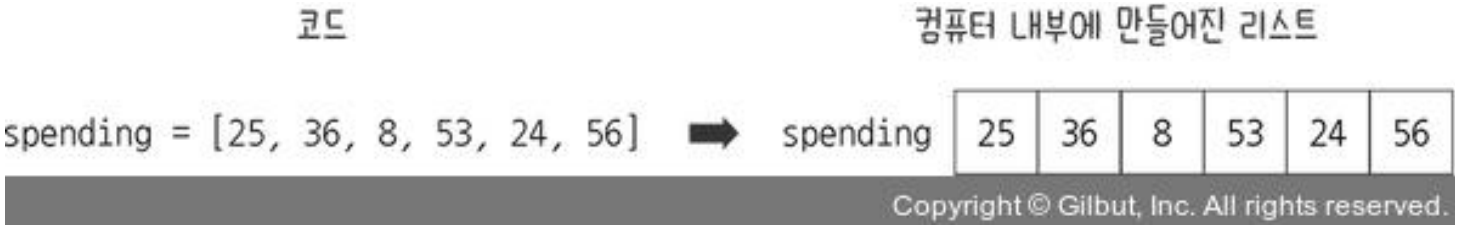
형식

```
리스트명 = [데이터, 데이터…]
```

간단하게 1월부터 6월까지 상반기 지출만 넣어 리스트로 만들어 봅시다.

```
spending = [25, 36, 8, 53, 24, 56]
```

이처럼 데이터를 묶어 대괄호(`[]`)에 넣으면 리스트가 됩니다(반복문을 배울 때 사용한 적이 있는데, 이것이 리스트였습니다). 리스트 안에 넣은 데이터는 요소(element) 또는 원소라고 하고, 각 요소는 콤마(`,`)로 구분합니다. 이를 `spending`이라는 이름의 변수에 담아 저장합니다. 코드를 실행하면 컴퓨터 내부에서는 다음 형태로 리스트가 만들어집니다.



Copyright © Gilbut, Inc. All rights reserved.

그럼 예제를 보며 리스트를 좀 더 자세히 알아봅시다.

리스트의 데이터 타입 확인하기

앞의 코드를 실행하면 아무것도 출력되지 않습니다. 이제는 왜 그런지 압니다. `print()`로 출력하지 않았기 때문이죠. 그럼 `abc`라는 리스트를 만들고 `print(abc)`로 출력해 봅시다.

```
abc = [1, 2, 3, 4, 5]
print(abc)
```

```
실행결과
[1, 2, 3, 4, 5]
```

리스트가 출력됩니다. 콤마로 구분된 5개 데이터와 대괄호까지 출력됩니다. 이처럼 리스트를 출력하면 각각의 데이터가 아니라 데이터를 대괄호로 묶어 전체를 보여 줍니다.

`type()`으로 `abc`의 데이터 타입을 출력해 봅시다.

```
abc = [1, 2, 3, 4, 5]
print(type(abc), abc)
```

```
실행결과
<class 'list'> [1, 2, 3, 4, 5]
```

데이터 타입과 리스트가 동시에 출력됩니다. `abc`는 리스트 타입이며 `[1, 2, 3, 4, 5]`를 저장하고 있다는 것을 보여 줍니다(3장에서 `class`는 '그룹'으로 이해하면 된다고 했으니 여기서도 리스트 그룹이라고 보면 됩니다). 그러므로 리스트도 정수형, 실수형, 문자열처럼 하나의 데이터 타입임을 알 수 있습니다.

len()으로 리스트 길이 알아내기

이번에는 처음에 예로 든 일년 지출과 상반기 지출을 각각 리스트로 만든 후 데이터 타입과 그 값을 출력해 봅시다.

```
# 일년 지출
spending_year = [25, 36, 8, 53, 24, 56, 38, 79, 94, 20, 40, 25]
print(type(spending_year), spending_year)
# 상반기 지출
spending_6mon = [25, 36, 8, 53, 24, 56]
print(type(spending_6mon), spending_6mon)
```

```
실행결과
<class 'list'> [25, 36, 8, 53, 24, 56, 38, 79, 94, 20, 40, 25]
<class 'list'> [25, 36, 8, 53, 24, 56]
```

`spending_year`와 `spending_6mon`에 각각 12개와 6개 데이터가 잘 들어갔는지 확인해 봅시다. 이럴 때 사용하기 좋은 명령어가 있습니다. 리스트 길이를 알려 주는 `len()`입니다. `len`은 length의 앞 세 글자를 딴 명령어로, `len(spending_year)`처럼 소괄호 안에 길이를 알고 싶은 리스트를 넣으면 됩니다.

```
# 일년 지출
spending_year = [25, 36, 8, 53, 24, 56, 38, 79, 94, 20, 40, 25]
print(len(spending_year))

# 상반기 지출
spending_6mon = [25, 36, 8, 53, 24, 56]
print(len(spending_6mon))
```

```
실행결과
12
6
```

결과로 12와 6이 나옵니다. 다른 숫자가 나온다면 데이터 개수가 제대로 입력되지 않았다는 뜻이니 코드를 다시 살펴보세요.

리스트가 어떤 것인지 조금 알게 됐으니 이번에는 숫자가 아닌 문자를 넣어 보겠습니다. 다음은 음식을 좋아하는 순서대로 리스트에 저장해 출력하는 코드입니다.

```
# 좋아하는 음식
food = ['피자', '치킨', '떡볶이', '스테이크', '딸기', '파스타']
print(food)
```

```
실행결과
['피자', '치킨', '떡볶이', '스테이크', '딸기', '파스타']
```

숫자와 마찬가지로 각 문자를 콤마(`,`)로 구분해 리스트에 넣었습니다. 이때 문자임을 표시하기 위해 작은따옴표로 감쌉니다. 좋아하는 순서대로 입력했으니 가장 좋아하는 음식은 `food` 리스트의 첫 번째 요소고, 두 번째로 좋아하는 음식은 `food` 리스트의 두 번째 요소겠죠.

`food` 리스트의 길이도 출력해 볼까요? 출력하는 내용이 뭔지 알 수 있게 '좋아하는 음식 개수:'와 '좋아하는 음식 순서:'를 넣어 출력하겠습니다.

```
# 좋아하는 음식
food = ['피자', '치킨', '떡볶이', '스테이크', '딸기', '파스타']
print('좋아하는 음식 개수:', len(food))
print('좋아하는 음식 순서:', food)
```

```
실행결과
좋아하는 음식 개수: 6
좋아하는 음식 순서: ['피자', '치킨', '떡볶이', '스테이크', '딸기', '파스타']
```

좋아하는 음식 개수가 6개네요. 이처럼 리스트를 다룰 때 `len()`을 자주 사용하니 사용법을 잘 익혀 두세요.

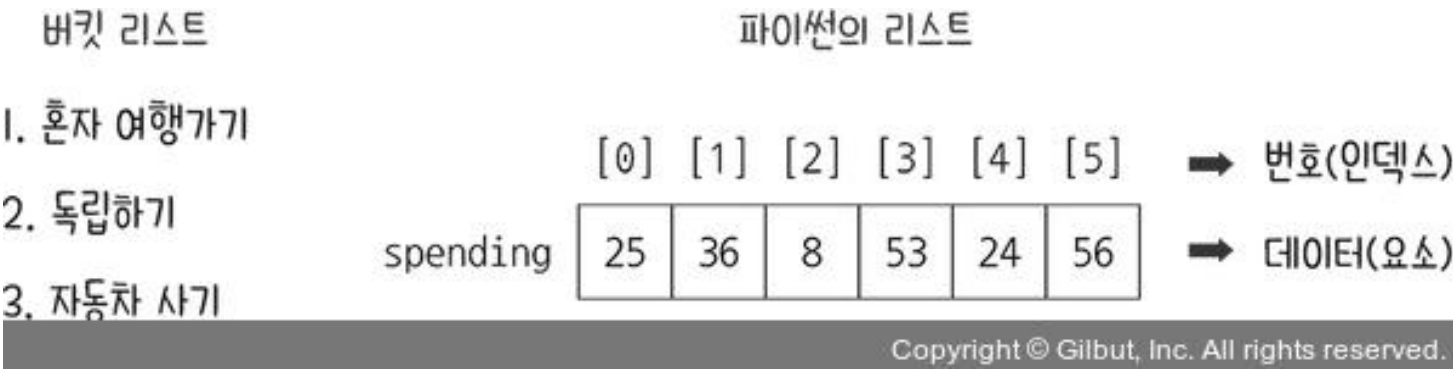
6.1.2 리스트 사용하기

리스트가 무엇이고 어떻게 만드는지 알아봤습니다. 이번에는 리스트의 데이터를 어떻게 사용하는지 알아보겠습니다.

리스트의 인덱스

리스트는 일상생활에서도 흔히 쓰는 단어입니다. 마트에 장 보러 갈 때 사야 할 물건 리스트를 작성하거나 해 보고 싶은 일을 버킷 리스트로 적기도 합니다. 리스트의 사전적 정의는 '물품이나 사람의 이름 따위를 일정한 순서로 적어 놓은 명단, 목록'입니다. 여기서 '일정한 순서로 적어 놓은'이라는 부분을 주목해 봅시다.

버킷 리스트는 순서를 나타내는 숫자를 적고, 가장 하고 싶은 일부터 나열합니다. 번호와 데이터가 대응된 형태죠. 파이썬의 리스트도 비슷합니다. 리스트의 요소에는 순서가 있고, 이 순서를 숫자로 표현할 수 있습니다. 다음을 보면 리스트 위에 번호가 있습니다. 이는 리스트 안에서 요소의 위치를 알려 주는 숫자로, 인덱스(index)라고 합니다.



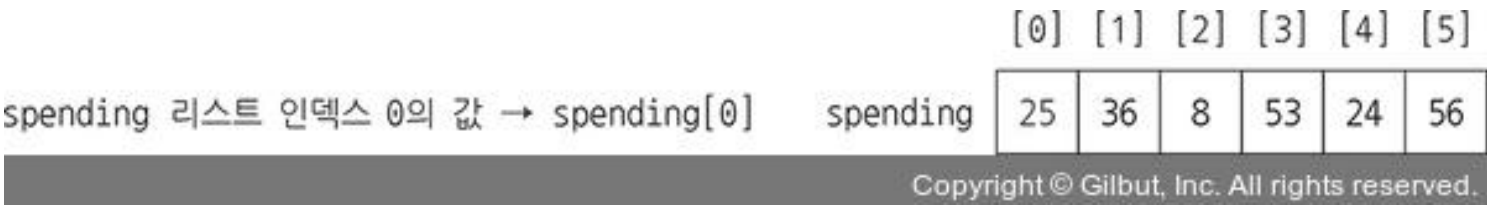
그래서 인덱스를 알면 리스트의 요소를 알 수 있습니다. 리스트의 요소에는 `리스트명[인덱스]` 형식으로 접근합니다. 즉, `spending[0]`은 25를, `spending[1]`은 36을, `spending[5]`는 56을 나타냅니다.

그런데 그림을 보면 인덱스가 1이 아닌 0부터 시작합니다. 실생활에서는 번호나 순서, 범위를 셀 때는 1부터 시작하지만, 컴퓨터에서는 0부터 시작합니다. 앞서 배운 `range()`도 시작 숫자를 생략하면 자동으로 시작 숫자를 0으로 간주했습니다. 리스트도 인덱스를 0부터 시작해서 요소가 6개 있으면 마지막 인덱스는 5가 됩니다(인덱스가 0부터 시작해서 편리한 부분이 있는데, 이는 뒤에서 다루겠습니다). 예를 들어, 다음과 같이 코드를 작성하면 '리스트 `spending`의 첫 번째 값을 출력해.'라는 뜻입니다.

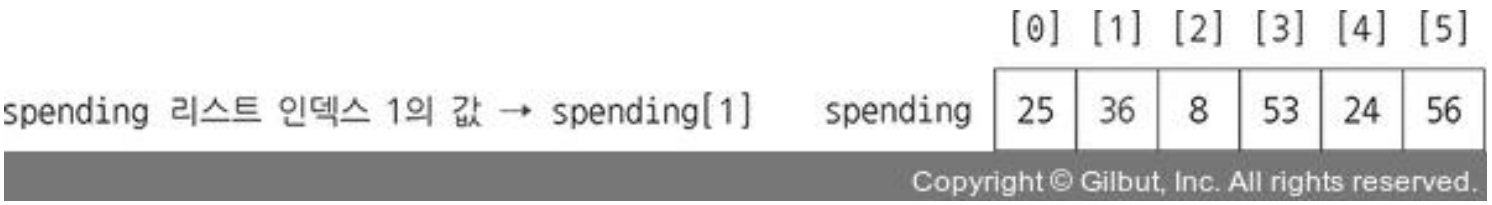
```
print(spending[0])
```

```
실행결과
25
```

리스트는 앞에서 보듯이 표현 방법이 매우 직관적입니다. 리스트명과 인덱스만 알면 되죠. 인덱스 1에 들어 있는 값은 `spending[1]`, 인덱스 2에 들어 있는 값은 `spending[2]`로 표현합니다.



Copyright © Gilbut, Inc. All rights reserved.



Copyright © Gilbut, Inc. All rights reserved.

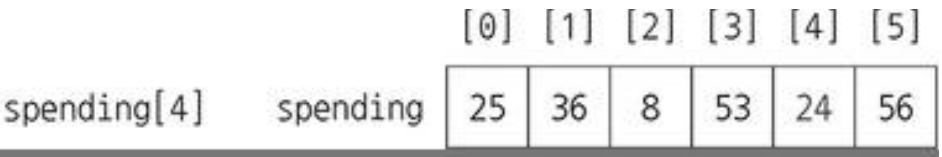


Copyright © Gilbut, Inc. All rights reserved.



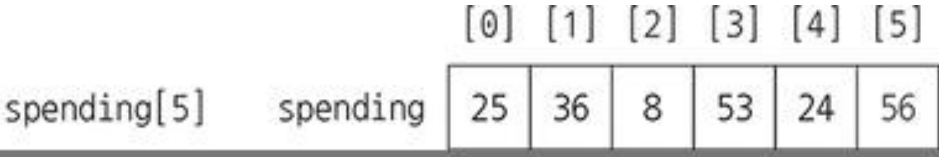
Copyright © Gilbut, Inc. All rights reserved.

spending 리스트 인덱스 4의 값 → spending[4]



Copyright © Gilbut, Inc. All rights reserved.

spending 리스트 인덱스 5의 값 → spending[5]



Copyright © Gilbut, Inc. All rights reserved.

코드로 직접 확인해 봅시다. 상반기 지출을 담은 `spending` 리스트를 다시 만들고 첫 번째 값을 출력해 봅시다.

```
spending = [25, 36, 8, 53, 24, 56]
print(spending[0])
```

```
실행결과
25
```

25가 출력됩니다. `spending`이 1월부터 6월까지의 지출을 차례대로 담은 리스트이므로 25는 1월 지출입니다. 인덱스가 0부터 시작하므로 `spending[0]`을 하면 1월 지출이 출력됩니다.

4월 지출은 어떻게 출력할까요? 앞의 코드에서 인덱스 숫자만 3으로 바꾸면 됩니다.

```
print(spending[3])
```

4월 지출은 `spending` 리스트의 네 번째 요소입니다. 따라서 `spending[3]`을 출력하면 4월 지출이 나옵니다.

만약 리스트에 없는 인덱스를 출력하면 어떻게 될까요? 다음 코드를 실행해 봅시다.

```
print(spending[6])
```

```
실행결과
----> 1 print(spending[6])
IndexError: list index out of range
```

`IndexError`가 발생합니다. 오류 메시지를 보면 리스트 인덱스가 범위를 벗어났다고 되어 있네요. 존재하지 않는 인덱스 `[6]`의 데이터에 접근하려고 해서 컴퓨터가 오류를 낸 겁니다. 이는 프로그래밍 입문자가 가장 흔하게 겪는 오류입니다. 인덱스를 살펴보면 금방 해결할 수 있으니 당황하지 마세요.

리스트의 값 변경하기

이번에는 리스트의 값을 바꿔 보겠습니다. 3월에 현금으로 사용한 10만 원이 누락되어 3월 지출에 10을 더해야 한다고 합시다. 3월 지출은 `spending[2]`에 있습니다. 여기에 10만 원을 더하면 되므로 `spending[2] = spending[2] + 10`으로 작성합니다. 이는 `spending[2] += 10`으로 줄여 쓸 수 있습니다. 변수에 값을 더하고 빼는 방법과 같죠. 추가로 변경 전과 변경 후의 값을 비교하기 위해 덧셈 연산 전후로 리스트 값을 확인할 수 있는 코드를 넣어 실행해 봅시다.

```
print('3월 지출 변경 전:', spending)
spending[2] += 10
print('3월 지출 변경 후:', spending)
```

```
실행결과
3월 지출 변경 전: [25, 36, 8, 53, 24, 56]
3월 지출 변경 후: [25, 36, 18, 53, 24, 56]
```

결과를 보면 세 번째 데이터가 8에서 18로 바뀌었습니다.

이번에는 5월 지출에 환불받은 2만 원이 포함됐다고 해 봅시다. 그럼 5월 지출에서 2를 빼야 합니다.

```
spending[4] -= 2
print('5월 지출 변경 후:', spending)
```

```
실행결과
5월 지출 변경 후: [25, 36, 18, 53, 22, 56]
```

리스트의 다섯 번째 요소가 24에서 22로 바뀌었습니다. 이처럼 리스트 안에 있는 개별 요소가 필요할 때는 해당 위치의 인덱스를 활용하면 됩니다.

6.1.3 반복문으로 리스트 다루기

이번에는 반복문으로 리스트의 요소를 출력해 보겠습니다.

리스트명 사용하기

반복문의 형식은 `for i in data`입니다. `data`에는 데이터 묶음이 들어가므로 리스트는 `data` 부분에 넣습니다. `spending` 리스트를 그대로 `data` 위치에 넣고 실행해 봅시다.

```
for i in [25, 36, 8, 53, 24, 56]:
   print(i)
```

```
실행결과
25
36
8
53
24
56
```

리스트의 첫 번째 데이터인 25부터 마지막 데이터인 56까지 차례대로 `i`에 들어가 `print(i)`가 실행됐습니다.

이제 데이터 묶음이 리스트라는 것을 알았으니 반복문에 데이터를 직접 넣는 대신에 리스트를 넣어도 됩니다. 반복문에서 리스트를 사용하면 보통 변수에 인덱스를 의미하는 `i` 대신 리스트의 첫 글자를 따서 넣습니다. 여기서도 `spending` 리스트의 앞 글자를 딴 `s`를 넣어 `for s in spending`로 작성해 보겠습니다.

`spending` 리스트를 다시 만든 다음 반복문에서 리스트를 사용하도록 코드를 바꿔 실행합니다.

```
spending = [25, 36, 8, 53, 24, 56]
for s in spending:
    print(s)
```

결과가 기존과 동일하게 나옵니다. `spending` 리스트에 `[25, 36, 8, 53, 24, 56]`이 담겨 있으니 컴퓨터는 `for s in spending`과 `for s in [25, 36, 8, 53, 24, 56]`이 같다고 인식합니다. 앞으로는 반복문에 데이터를 일일이 넣는 대신 리스트명을 넣어 리스트의 요소에 접근하면 됩니다.

인덱스 사용하기

이번에는 인덱스로 리스트의 요소를 출력해 보겠습니다. `spending` 리스트의 요소는 `spending[0]`, `spending[1]`…`spending[5]`로 접근할 수 있습니다. 반복되는 부분과 변하는 부분이 보이나요? `spending[]`은 6번 반복되고 대괄호 안의 숫자인 인덱스만 0부터 5까지 1씩 증가합니다. 따라서 반복문에서 배운 `range()`를 사용해 다음처럼 코드를 작성할 수 있습니다.

```
spending = [25, 36, 8, 53, 24, 56]
for i in range(6):
    print(spending[i])
```

리스트를 반복문에서 활용할 때 `for a in` 리스트처럼 리스트를 반복문에 직접 넣을 수도 있고(이때 `a`는 리스트명의 첫 글자), `for i in range()`처럼 인덱스를 활용할 수도 있습니다. 선택한 방법에 따라 반복해서 실행되는 코드가 조금 달라집니다.

이번에는 좀 더 정보를 보여 줄 수 있게 '1월 지출은 25' 형태로 출력해 보겠습니다. 먼저 월은 1부터 6까지 1씩 증가합니다. 0부터 5까지 증가하는 인덱스보다 월이 항상 1만큼 큽니다. 따라서 출력 부분은 `print(i, '월 지출은')`이 아닌 `print(i + 1, '월 지출은')`이 돼야 합니다. 이를 기존 반복문에 추가해 실행해 봅시다.

```
for i in range(6):
    print(i + 1,'월 지출은', spending[i])
```

```
실행결과
1 월 지출은 25
2 월 지출은 36
3 월 지출은 8
4 월 지출은 53
5 월 지출은 24
6 월 지출은 56
```

이처럼 인덱스를 활용할 때는 인덱스가 0부터 시작한다는 점을 유의해야 합니다.

이번에는 반복문에서 리스트를 사용할 때 입문자가 자주 겪는 또 다른 오류 상황을 살펴보겠습니다. 리스트를 사용하다 보면 리스트의 요소 개수를 헷갈릴 때가 종종 있습니다. 다음처럼 `range()` 안의 숫자를 7로 바꾸고 실행해 봅시다.

```
for i in range(7):
    print(i + 1,'월 지출은', spending[i])
```

```
실행결과
1 월 지출은 25
2 월 지출은 36
3 월 지출은 8
4 월 지출은 53
5 월 지출은 24
6 월 지출은 56
---------------------------------------------------------------------------
      1 for i in range(7):
----> 2 print(i + 1, '월 지출은', spending[i])
IndexError: list index out of range
```

`i`가 0부터 5일 때까지 정상적으로 출력되다가 6이 될 때 `IndexError`가 발생합니다. 인덱스는 6개인데 이를 초과한 7로 범위를 설정해서 그렇습니다. 지금처럼 코드가 짧을 때는 리스트 길이를 기억해 이런 오류가 거의 발생하지 않습니다. 하지만 리스트가 여러 개거나 데이터가 많으면 리스트 길이를 일일이 기억하기가 어렵습니다. 이럴 때 앞에서 나온 `len()`을 사용하면 됩니다.

리스트의 인덱스는 항상 0부터 시작하기 때문에 인덱스 범위는 0~(리스트 길이 - 1)입니다. `spending` 리스트도 길이는 6이지만, 인덱스는 0부터 5까지입니다. 이를 `range()` 명령어에 활용할 수 있습니다. `range(종료 숫자)`는 자동으로 0부터 (종료 숫자 - 1) 사이의 연속된 숫자를 의미했죠. 이때 종료 숫자 대신 리스트 길이를 넣으면 어떻게 될까요? 코드로 작성해 봅시다.

```
for i in range(len(spending)):
    print(i + 1, '월 지출은', spending[i], end=', ')
```

```
실행결과
1 월 지출은 25, 2 월 지출은 36, 3 월 지출은 8, 4 월 지출은 53, 5 월 지출은 24, 6 월 지출은 56
```

인덱스로 범위를 정했을 때와 마찬가지로 1월부터 6월까지 지출이 출력됩니다. 이처럼 리스트의 첫 인덱스부터 마지막 인덱스까지 반복 범위를 설정하고 싶을 때, 인덱스를 입력하지 않고도 `range(len(리스트명))`으로 리스트 안의 모든 데이터에 접근할 수 있습니다.

새로운 리스트를 만들어 봅시다. 앞에서처럼 좋아하는 음식을 순서대로 저장한 `food` 리스트를 만듭니다. 이 리스트에서 가장 좋아하는 음식, 세 번째로 좋아하는 음식, 가장 덜 좋아하는 음식을 출력해 봅시다. 직접 코드를 작성하고 책과 비교해 보세요. 앞에서 배운 내용이니 작성하기 어렵지 않을 겁니다.

```
food = ['피자', '치킨', '떡볶이', '스테이크', '파스타']
# 가장 좋아하는 음식은?
print(food[0])
# 세 번째로 좋아하는 음식은?
print(food[2])
# 가장 덜 좋아하는 음식은?
print(food[4])
```

```
실행결과
피자
떡볶이
파스타
```

가장 좋아하는 음식은 `food[0]`을, 세 번째로 좋아하는 음식은 `food[2]`를, 가장 덜 좋아하는 음식은 마지막 데이터이므로 `food[4]`를 출력합니다.

그런데 리스트의 마지막 인덱스는 (리스트 길이 - 1)이었죠. 그렇다면 가장 덜 좋아하는 음식을 출력할 때 인덱스를 직접 넣는 대신 (리스트 길이 - 1)로 바꿔 봅시다.

```
# 가장 덜 좋아하는 음식은?
print(food[len(food) - 1])
```

```
실행결과
파스타
```

이처럼 인덱스 위치에 식이 들어가도 됩니다. 인덱스 역시 숫자이므로 연산이 가능한 수식이 들어가도 정상적으로 실행됩니다.

음수 인덱스 사용하기

지금까지는 인덱스가 모두 양수였습니다. 양수 인덱스는 왼쪽부터 시작해 오른쪽으로 1씩 증가합니다. 그런데 인덱스에 음수도 들어갈 수 있습니다. 음수 인덱스는 오른쪽에서 시작해 왼쪽으로 1씩 감소합니다.



리스트 위에 있는 숫자는 지금까지 배운 양수 인덱스입니다. 리스트 아래 있는 숫자를 봅시다. 가장 오른쪽 숫자는 -1이고, 왼쪽으로 갈수록 1씩 감소해 -2, -3, -4, -5가 됩니다.

코드로 확인해 봅시다. 가장 덜 좋아하는 음식은 `food[4]`였으니 음수 인덱스로 `food[-1]`을 출력하면 됩니다.

```
# 가장 덜 좋아하는 음식은?
print(food[-1])
```

실행결과

```
파스타
```

`food` 리스트의 마지막에 있는 파스타가 출력됩니다. 두 번째로 덜 좋아하는 음식과 세 번째로 덜 좋아하는 음식도 출력해 봅시다.

```
# 두 번째로 덜 좋아하는 음식?
print(food[-2])
# 세 번째로 덜 좋아하는 음식?
print(food[-3])
```

```
실행결과
스테이크
떡볶이
```

`food[-2]`는 뒤에서 두 번째 데이터를 의미하므로 '스테이크'가 출력되고, `food[-3]`은 뒤에서 세 번째 데이터인 '떡볶이'를 출력합니다. 지금은 음수 인덱스의 유용성이 와닿지 않겠지만, 방대한 데이터를 분석할 때 큰 위력을 발휘하니 잘 알아 두기 바랍니다.

> NOTE 음수 인덱스는 왜 0부터 시작하지 않을까요?
>
> 인덱스에 0을 2개나 쓸 수 없기 때문입니다. 음수 인덱스가 0부터 시작한다면 첫 번째 데이터와 마지막 데이터 모두 인덱스가 0이 되겠죠. 그러면 컴퓨터는 인덱스 0이 어떤 데이터를 의미하는지 헷갈릴 수밖에 없습니다. 그래서 왼쪽에서 시작하는 인덱스는 0부터, 오른쪽에서 시작하는 음수 인덱스는 -1부터 시작합니다.

6.1.4 리스트에 요소 추가하고 삭제하기

리스트에 요소를 추가하고 삭제해 보겠습니다.

요소 추가하기

추가하는 방법부터 살펴봅시다. 리스트에 요소를 추가할 때는 `append()` 명령어를 사용합니다. `append()` 명령어는 다음과 같이 사용합니다.

형식

```
리스트명.append(데이터)
```

명령어 형식도 일종의 규칙이므로 잘 기억해 두세요. '`food` 리스트에 '감자칩'을 추가하라'고 하고 싶으면 `food.append('감자칩')`을 실행하면 됩니다. 참고로 append는 '덧붙이다'는 의미가 있으니 단어의 의미를 생각하면 어떤 일을 하는지 이해하기 쉽습니다.



바로 확인해 봅시다.

```
food = ['피자', '치킨', '떡볶이', '스테이크', '파스타']
food.append('감자칩')
print(food)
```

실행결과

```
['피자', '치킨', '떡볶이', '스테이크', '파스타', '감자칩']
```

`food`는 처음에 데이터가 5개인 리스트였습니다. `food.append('감자칩')`을 실행하고 나면 리스트의 맨 뒤에 감자칩이 추가되어 데이터는 6개가 됩니다. 데이터를 하나 더 추가해 볼까요? '카레'를 추가해 봅시다.

```
food.append('카레')
print(food)
```

```
실행결과
['피자', '치킨', '떡볶이', '스테이크', '파스타', '감자칩', '카레']
```

`food` 리스트 맨 뒤에 카레가 추가됐습니다. 이처럼 `append()` 명령어는 리스트의 맨 뒤에 데이터를 추가합니다. `append()`가 실행될 때 컴퓨터 내부에서는 다음과 같은 일이 일어납니다.



Copyright © Gilbut, Inc. All rights reserved.

리스트에 데이터가 추가되면 리스트 길이도 늘어나고 인덱스도 추가됩니다. 또한, 빈 리스트를 만들어 데이터를 추가할 수도 있습니다. 다음 코드를 봅시다.

```
abc = [] # 빈 리스트 만들기
abc.append(24)
abc.append(38)
abc.append(19)
abc.append(40)
print(abc)
```

```
실행결과
[24, 38, 19, 40]
```

`abc = []`처럼 리스트명에 아무런 데이터 없이 대괄호만 넣으면 빈 리스트가 생성됩니다. 그다음 `append()`로 빈 리스트에 24, 38, 18, 40을 순서대로 추가하면 됩니다.

직접 값을 입력받아 리스트에 넣을 수도 있습니다. 해야 할 일을 입력받아 `to_do` 리스트에 저장해 봅시다. 어떤 값이 입력될지 모르니 데이터가 없는 빈 리스트를 만듭니다. `input()`으로 해야 할 일을 입력받고 `append()`로 리스트에 추가합니다.

```
to_do = []
task = input('해야 할 일을 입력해 주세요. --> ')
to_do.append(task)
print(to_do)
```

```
실행결과
해야 할 일을 입력해 주세요. --> 파이썬 공부하기
['파이썬 공부하기']
```

`to_do`라는 이름으로 빈 리스트를 만들고 해야 할 일을 입력받아 변수 `task`에 저장합니다. 그리고 `task`를 `append()` 명령어로 `to_do`에 추가합니다(`to_do`와 `append()` 사이에 도트(`.`)가 있음에 유의하세요). 이때 `task` 변수를 사용하지 않고 바로 `input()`으로 입력받은 내용을 `append()` 명령어에 넣어도 됩니다.

```
to_do = []
to_do.append(input('해야 할 일을 입력해 주세요. --> '))
print(to_do)
```

```
실행결과
해야 할 일을 입력해 주세요. --> 파이썬 공부하기
['파이썬 공부하기']
```

이번에는 해야 할 일 5개를 입력받아 `to_do` 리스트에 추가해 보겠습니다. 입력을 5번 반복해야 하니 반복문을 사용합니다.

```
to_do = []
print('해야 할 일을 차례대로 입력해 주세요.')
for i in range(5):
    to_do.append(input())
print(to_do)
```

```
실행결과
해야 할 일을 차례대로 입력해 주세요.
생일선물 사기
인강 듣기
파이썬 6장 공부하기
책 읽기
영단어 외우기
['생일선물 사기', '인강 듣기', '파이썬 6장 공부하기', '책 읽기', '영단어 외우기']
```

먼저 빈 리스트를 만듭니다. 리스트를 만들지 않은 채 데이터를 추가하면 오류가 발생합니다(물론 기존 셀의 결괏값이 남아 있는 경우에는 오류가 발생하지 않지만, 단독 코드일 때는 오류가 발생합니다). `to_do.append(input())`으로 입력받은 값을 변수에 따로 저장하지 않고 바로 리스트에 추가합니다. 괄호 안의 `input()`을 먼저 실행하고 나서 `to_do.append()`를 실행하므로 입력받은 값을 바로 `to_do` 리스트에 추가할 수 있습니다. 입력을 5번 받아야 하므로 이 코드를 반복문 안에 넣습니다.

반복문 안에 `print()` 명령어를 넣어 반복문이 진행되는 동안 리스트가 어떻게 변하는지 확인해 보세요.

```
to_do = []
print('해야 할 일을 차례대로 입력해 주세요.')
for i in range(5):
    to_do.append(input())
    print(to_do)
```

반복할 때마다 리스트 맨 뒤에 입력한 데이터가 추가될 겁니다.

요소 삭제하기

리스트에 있는 데이터를 삭제하는 방법도 배워 보겠습니다. 리스트의 데이터를 삭제하는 방법은 크게 두 가지가 있습니다. 먼저 리스트에 추가할 때처럼 리스트명 뒤에 `remove(데이터)`를 붙여 리스트의 데이터를 삭제할 수 있습니다.

형식

```
리스트명.remove(데이터)
```

`food` 리스트에서 떡볶이만 삭제해 봅시다.

```
food = ['피자', '치킨', '떡볶이', '스테이크', '파스타']
food.remove('떡볶이')
print(food)
```

실행결과

```
['피자', '치킨', '스테이크', '파스타']
```

이처럼 삭제할 데이터를 `remove()`를 사용해 직접 삭제합니다. 또는 인덱스를 사용해서 데이터를 삭제할 수 있습니다. 형식은 다음과 같습니다.

형식

```
del 리스트명[인덱스]
```

`food` 리스트에서 가장 마지막에 있는 음식을 지워 볼까요?

```
food = ['피자', '치킨', '떡볶이', '스테이크', '파스타']
del food[-1]
print(food)
```

`실행결과`

```
['피자', '치킨', '떡볶이', '스테이크']
```

음수 인덱스로 가장 마지막에 있는 파스타를 삭제했습니다.

6.1.5 리스트에서 필요한 요소만 꺼내기

지금까지 사용한 리스트는 매우 간단했습니다. 그러나 리스트에 더 많은 요소를 담을 수도 있습니다. 많은 요소가 있으면 때때로 필요한 부분과 필요하지 않은 부분이 섞일 수 있습니다. 그래서 원하는 정보를 얻으려면 필요한 부분만 꺼내 쓸 수 있어야 합니다.

리스트 슬라이싱하기

연, 월, 일, 시, 분을 `[2020, 4, 5, 11, 30]` 형태로 저장한 리스트가 있다고 가정해 봅시다. 이 리스트에서 월, 일만 필요하다면 인덱스 1과 2의 데이터만 추출해야 합니다. 또는 날짜를 제외한 시간 데이터만 필요한 경우에는 마지막 요소 2개만 골라내야죠. 앞서 사용한 `food` 리스트에서도 가장 좋아하는 음식 3개나 가장 덜 좋아하는 음식 2개 등 원하는 부분만 추출해야 할 수도 있습니다.

이처럼 원하는 부분만 골라내는 것을 리스트 슬라이싱이라고 합니다. 슬라이싱(slicing)은 (얇게) 자르는 것을 말합니다. 리스트를 자르는 게 어떤 건지 이해하기 쉽게 예를 들어 보겠습니다. 보통 '1일부터 5일까지'를 간략하게 '1~5일'로 물결표(~)를 써서 나타냅니다. 리스트 슬라이싱도 이와 비슷합니다. '여기에서 저기까지 데이터를 뽑아줘'라는 뜻입니다. '여기'와 '저기'는 데이터의 위치를 나타내니 인덱스가 필요하겠죠. 그럼 '인덱스 a에서 b까지 데이터를 뽑아줘'라고 표현할 수 있습니다. 물결표를 사용해 더 간단히 나타내면 '인덱스 a~b', 더 줄여서 [a~b]라고 표현할 수 있습니다. 다만, 파이썬에서는 물결표 대신 콜론(:)을 사용합니다. 예를 들어, '`food` 리스트의 인덱스 0부터 2까지'를 표현하면 `food[0:3]`이 됩니다.

잠깐만요! 왜 `food[0:2]`가 아닌 `food[0:3]`일까요? `range(a, b)` 명령어를 생각해 보세요. `range(a, b)` 명령어처럼 `리스트명[a:b]`에서도 `a`는 시작 위치, `b`는 종료 위치입니다. `b`까지 실행되는 것이 아니라 `b-1`까지만 실행됩니다. 실제로는 `b`가 아닌 `b-1` 인덱스까지만 포함하는 것이죠.



Copyright © Gilbut, Inc. All rights reserved.

코드로 확인해 봅시다.

```
food = ['피자', '치킨', '떡볶이', '스테이크', '파스타']
print(food[0:3])
```

```
실행결과
['피자', '치킨', '떡볶이']
```

`food[0:3]`은 `food[0]`부터 f`ood[2]`까지를 의미하므로 `food` 리스트에서 앞부터 3개의 데이터, 즉 피자부터 떡볶이까지만 출력됩니다.

이번에는 떡볶이부터 스테이크까지 출력하는 코드를 작성해 볼까요? 떡볶이는 `food[2]`에 있고, 스테이크는 `food[3]`에 있으니 `food[2:4]`로 슬라이싱하면 됩니다.

```
print(food[2:4])
```

```
실행결과
['떡볶이', '스테이크']
```

리스트 슬라이싱할 때 `range()`와 비슷해서 시작 인덱스가 0이면 생략할 수 있습니다. 따라서 `food[0:3]`에서 0을 생략하고 `food[:3]`으로도 사용합니다.

```
print(food[:3])
```

```
실행결과
['피자', '치킨', '떡볶이']
```

`food[0:3]`일 때와 같은 결과가 나옵니다.

그런데 리스트 슬라이싱에서는 `range()`와 달리 종료 인덱스도 생략할 수 있습니다. 종료 인덱스를 생략하면 시작 위치부터 리스트 끝까지 데이터를 슬라이싱합니다. 가장 덜 좋아하는 음식 2개를 추출하는 코드를 작성해 보겠습니다. `food` 리스트의 맨 뒤 데이터 2개를 선택하면 되겠죠. 뒤에서 세는 것이 편하니 음수 인덱스를 사용해 봅시다. 시작 위치는 뒤에서 두 번째이므로 `[-2]`가 되고 종료 인덱스는 생략해도 되니 `food[-2:]`면 됩니다.

```
print(food[-2:])
```

```
실행결과
['스테이크', '파스타']
```

뒤에서 두 번째 데이터인 스테이크부터 마지막 데이터인 파스타까지 출력됩니다.

리스트 슬라이싱의 생략 규칙을 정리하면 다음과 같습니다.



Copyright © Gilbut, Inc. All rights reserved.



Copyright © Gilbut, Inc. All rights reserved.

앞에서 사용한 상반기 지출 리스트로 리스트 슬라이싱해 봅시다. 1월부터 3월까지, 3월부터 5월까지, 5월부터 6월까지 지출을 각각 출력해 봅시다.

```
spending_6mon = [25, 36, 8, 53, 24, 56]
print('1월부터 3월 지출:', spending_6mon[:3])
print('3월부터 5월 지출:', spending_6mon[2:5])
print('5월부터 6월 지출:', spending_6mon[-2:])
```

```
실행결과
1월부터 3월 지출: [25, 36, 8]
3월부터 5월 지출: [8, 53, 24]
5월부터 6월 지출: [24, 56]
```

1~3월까지 지출은 `spending_6mon` 리스트의 인덱스 `[0]`부터 `[2]`까지이므로 `spending_6mon[0:3]`이 됩니다. 시작 인덱스가 0이라서 이를 생략하고 `spending_6mon[:3]`으로 작성합니다. 3~5월 지출액만 슬라이싱하려면 리스트의 인덱스 `[2]`부터 `[4]`까지 추출하면 됩니다. 따라서 `spending_6mon[2:5]`가 됩니다. 5~6월은 `spending_6mon` 리스트의 뒤에서 데이터 2개를 뽑는 것과 같습니다. 5월은 뒤에서 두 번째 데이터이므로 음수 인덱스를 사용하고, 종료 인덱스를 생략하면 `spending_6mon[-2:]`가 됩니다.

문자열 슬라이싱하기
문자열에서도 리스트와 마찬가지 슬라이싱할 수 있습니다. 문자열은 어떻게 슬라이싱할까요? 한 번 알아봅시다.

문자열도 리스트처럼 글자에 번호를 붙일 수 있습니다. '나는야 우주최강'이라는 문자열에 번호를 붙이면 다음과 같습니다. 이때 띄어쓰기로 생긴 공백도 문자열에 포함되므로 번호를 붙입니다.

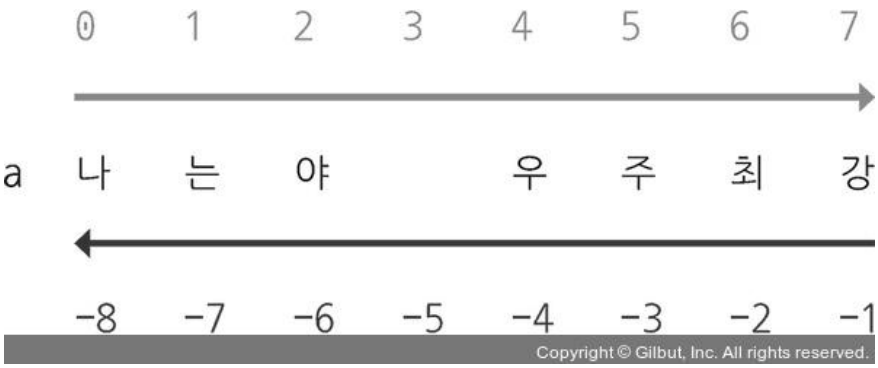


문자열 슬라이싱은 리스트와 형식이 같습니다. 문자열이 `a` 변수에 저장되어 있을 때 앞의 세 글자인 '나는야'만 슬라이싱해 봅시다. 형식이 같으니 `a[0:3]` 또는 `a[:3]`이면 되겠죠? 코드로 확인해 봅시다.

```
a = '나는야 우주최강'
print(a[:3])
```

```
실행결과
나는야
```

3번째 글자까지 슬라이싱이 잘 됐습니다. 이번에는 '우주최강'을 슬라이싱해 봅시다. 뒤에서 네 글자만 골라내면 됩니다. 뒤에서부터 세므로 음수 인덱스를 사용해 `a[-4:]`로 작성하면 됩니다.




```
print(a[-4:])
```

```
실행결과
우주최강
```

이번에는 '우주'만 뽑아봅시다. 어떻게 하면 될까요? 책을 보고 그대로 작성하기 전에 스스로 생각해 봅시다! 그다음 코드를 작성하고 자신의 생각이 맞는지 책을 확인하세요.

```
print(a[-4:-2])
```

```
실행결과
우주
```

책처럼 음수 인덱스를 사용해도 되고, 양수 인덱스를 사용해도 됩니다. 바로 앞에 음수 인덱스가 나와서 그대로 음수 인덱스를 사용했습니다. 리스트 슬라이싱을 사용할 때 `리스트[a:b]`에서 `[b]`는 종료 인덱스이므로 `[b-1]`까지 슬라이싱됩니다. 따라서 '우주'를 출력하려면 `a[-4:-2]`로 작성해야 합니다.

다른 문자열로 확인해 봅시다. '2021-04-15'라는 날짜를 `date` 변수에 넣은 뒤 연도, 월, 일을 각각 슬라이싱해 봅시다.

```
date = '2021-04-15'
print('연도:', date[:4])
print('월:', date[5:7])
print('일:', date[-2:])
```

```
실행결과
연도: 2021
월: 04
일: 15
```

글자 위치에 맞춰 양수든 음수든 편한 쪽을 사용하면 됩니다.

split()으로 문자열 나누기

문자열을 슬라이싱할 때 인덱스를 사용하면 간단하기는 하지만, 인덱스를 일일이 세야 하기 때문에 문자열이 길어지면 결괏값이 잘못 나올 수도 있습니다. 이럴 때 유용한 `split()` 명령어가 있습니다. split에는 '나누다, 찢다'는 의미가 있습니다. 단어의 의미처럼 `split()`은 지정한 글자를 기준으로 좌우로 문자를 나눕니다. 형식은 리스트에서 명령어를 사용할 때와 비슷합니다.

형식

```
문자열.split()
```

다음 코드를 실행해 봅시다.

```
a = '나는야 우주최강'
print(a.split())
```

```
실행결과
['나는야', '우주최강']
```

`split()`의 소괄호 안에 아무것도 지정하지 않으면 공백을 기준으로 나눕니다. 그런데 실행결과가 리스트로 나옵니다. `split()` 명령어를 사용하면 데이터가 1개에서 여러 개로 나뉘므로 데이터 묶음인 리스트로 출력합니다.

`a.split()`은 결괏값이 리스트이므로 인덱스를 붙일 수 있습니다. 확인해 봅시다.

```
a = '나는야 우주최강'
print(a.split())
print(a.split()[0])
print(a.split()[1])
```

```
실행결과
['나는야', '우주최강']
나는야
우주최강
```

이처럼 `split()`을 실행한 결괏값에 인덱스로 접근할 수 있습니다.

날짜가 들어 있는 `date` 변수도 `split()`으로 나눠 봅시다. `date`에는 공백이 없지만, 그 대신 하이픈(`-`)이 있으니 하이픈을 기준으로 나눠 보겠습니다. 나누는 글자를 지정하고 싶을 때는 `split()` 명령어 안에 넣으면 됩니다. 하이픈은 문자열이므로 양옆을 작은따옴표로 감싸야 합니다.

```
date = '2021-04-15'
print(date.split('-'))
```

```
실행결과
['2021', '04', '15']
```

`date`에는 하이픈(`-`)이 2개 있어서 하이픈을 기준으로 나누면 세 부분으로 나뉩니다. 공백을 기준으로 나눴을 때처럼 하이픈은 사라지고 연, 월, 일 숫자만 결과 리스트에 들어갑니다.

이번에는 인덱스를 사용해 연, 월, 일을 각각 출력해 봅시다.

```
date = '2021-04-15'
print(date.split('-')[0])
print(date.split('-')[1])
print(date.split('-')[2])
```

```
실행결과
2021
04
15
```

코드가 중복되긴 하지만, 인덱스를 일일이 세지 않아도 돼서 편합니다.

6.1.6 다양한 리스트 명령어 사용하기

리스트에는 정말 많은 기능이 있습니다. 이 책에서 전부 배우면 좋겠지만, 시작 단계에서 너무 많은 지식을 배우면 오히려 지칠 뿐입니다. 그래서 여기서는 몇 가지 기능만 간단히 짚어보겠습니다.

정렬하기

리스트에 들어 있는 데이터를 일정한 기준에 따라 정렬할 수 있는데, 이때 명령어 `sort()`를 사용합니다. 사용법은 `append()`와 같습니다.

형식

```
리스트명.sort()
```

예를 들어 `food` 리스트를 정렬하고 싶다면 `food.sort()`라고 하면 됩니다.

```
food = ['피자', '치킨', '스테이크', '된장찌개', '바나나', '파스타', '감자칩']
food.sort()
print('정렬 후:', food)
```

실행결과

```
정렬 후: ['감자칩', '된장찌개', '바나나', '스테이크', '치킨', '파스타', '피자']
```

`food` 리스트에 데이터 7개를 넣고 `food.sort()`로 정렬하니 맨 뒤에 있던 감자칩이 첫 번째 데이터가 됐습니다. `sort()` 명령어를 사용할 때 아무것도 지정하지 않으면 이처럼 오름차순으로 데이터를 정렬합니다. 여기서는 한글의 가나다순으로 데이터를 정렬했습니다.

반대로 내림차순으로 정렬하려면 `sort()` 명령어에 `reverse=True` 옵션을 추가하면 됩니다. `reverse`는 '뒤집다, 반전시키다' 등의 의미가 있으니 의미를 생각하면 이해하기 쉬울 거예요. 단어 의미 그대로 `reverse=True`는 '뒤집기=참'으로 보면 됩니다. 그리고 `True`는 첫 글자만 대문자임에 유의하세요!

옵션을 추가하고 실행해 봅시다.

```
food = ['피자', '치킨', '스테이크', '된장찌개', '바나나', '파스타', '감자칩']
food.sort(reverse=True)
print('정렬 후:', food)
```

```
실행결과
정렬 후: ['피자', '파스타', '치킨', '스테이크', '바나나', '된장찌개', '감자칩']
```

앞의 실행결과와 반대로 피자가 맨 앞으로 왔습니다. 만약 옵션을 `reverse=False`로 하면 어떨까요? '뒤집기=거짓'이므로 순서를 반전하지 않겠다는 뜻이 되어 오름차순으로 데이터를 정렬합니다. 그래서 오름차순 정렬일 때는 `reverse` 옵션을 생략하고 `food.sort()`만 해도 됩니다.

이번에는 상반기 지출 중 가장 큰 금액 2개를 뽑아 보겠습니다. 배운 내용을 중심으로 생각해 봅시다. 가장 큰 금액과 두 번째로 큰 금액을 찾으면 됩니다. 지출을 큰 금액부터 작은 금액순으로(내림차순) 정렬한 후 앞의 2개 데이터 뽑아내면 될 것 같습니다. 코드를 작성해 봅시다.

```
spending_6mon = [25, 36, 8, 53, 24, 56]
spending_6mon.sort(reverse=True)
print(spending_6mon[0:2])
```

```
실행결과
[56, 53]
```

내림차순으로 정렬해야 하니 `sort()` 명령어에 `reverse=True` 옵션을 추가합니다. 그런 다음 앞의 2개 데이터를 슬라이싱하면 가장 큰 금액 2개가 나옵니다.

그럼 가장 적은 금액 2개도 알아볼까요? 내림차순으로 정렬했으므로 가장 적은 금액은 리스트의 맨 뒤에 있을 겁니다. 뒤에서 두 번째 인덱스인 `[-2]`부터 끝까지 슬라이싱해 봅시다.

```
spending_6mon = [25, 36, 8, 53, 24, 56]
spending_6mon.sort(reverse=True)
print(spending_6mon[-2:])
```

```
실행결과
[24, 8]
```

이처럼 리스트의 기능을 복합적으로 사용하면 원하는 결과를 빠르게 얻을 수 있습니다. 데이터가 많은 리스트일수록 더욱더 그렇습니다.

최댓값과 최솟값, 합 구하기

앞에서처럼 리스트를 정렬하고 슬라이싱하면 리스트의 최댓값과 최솟값을 구할 수 있습니다. 그런데 리스트를 정렬해 슬라이싱하지 않고도 간단한 명령어로 최댓값과 최솟값을 구할 수 있습니다. 리스트의 최댓값을 구하는 명령어는 `max()`, 최솟값을 구하는 명령어 `min()`입니다. 코드로 사용법을 확인해 봅시다.

```
spending_6mon = [25, 36, 8, 53, 24, 56]
print(max(spending_6mon), min(spending_6mon))
```

```
실행결과
56, 8
```

`max()`와 `min()`은 리스트와 명령어를 도트(`.`)로 연결하지 않고 명령어 안에 리스트를 넣는 형태로 사용합니다. `len()`과 같은 형식이죠. 이 부분만 유의하세요.

리스트 데이터의 합을 구하는 `sum()` 명령어도 있습니다.

```
print(sum(spending_6mon))
```

```
실행결과
202
```

인덱스 찾기

마지막 기능은 인덱스를 찾는 `index()` 명령어입니다. 찾으려는 데이터를 `index()` 명령어의 소괄호 안에 넣으면 해당 데이터가 리스트의 어느 위치에 있는지, 즉 인덱스 번호를 알려줍니다. 예를 들어, `spending_6mon` 리스트에서 숫자 24의 인덱스를 알고 싶다면 다음과 같이 작성하면 됩니다.

```
print(spending_6mon.index(24))
```

```
실행결과
4
```

결과로 4가 나옵니다. 인덱스 `[4]`를 뜻하므로 실제로는 4에 1을 더한 5월 지출액을 의미합니다.

여기서 설명한 기능만 알고 있어도 리스트를 사용한 대부분의 코드를 이해할 수 있고, 리스트를 충분히 다룰 수 있습니다. 프로그래밍하다가 리스트와 관련한 다른 기능이 필요하다면 인터넷에서 검색해 보길 추천합니다.

1분 퀴즈

1 다음 중 옳지 않은 것을 고르세요.

① 리스트는 여러 개의 요소를 모아 놓은 데이터 묶음으로, `[]`를 사용해 표현한다.

② `a` 리스트에 새로운 요소를 추가하려면 `a.append()` 명령어를 사용한다.

③ 범위를 지정해 리스트의 요소를 추출할 수 있는데, 이를 리스트 슬라이싱이라고 한다.

④ 반복문으로 리스트의 전체 요소를 차례대로 출력할 수 있다.

⑤ `a` 리스트의 최댓값, 최솟값을 구하는 명령어는 `a.max()`, `a.min()`이다.

정답 및 해설

2 다음과 같은 리스트가 있을 때, 이와 관련해 옳지 않은 설명을 고르세요.

```
menu = ['아메리카노', '라떼', '바닐라라떼', '카푸치노', '핸드드립']
```

① 마지막 메뉴인 핸드드립을 출력하려면 `print(menu[-1])`을 실행한다.

② 새로운 메뉴를 사용자로부터 직접 입력받아 `menu`에 추가하려면 `menu.append(input('신메뉴를 입력하세요.'))`라고 작성한다.

③ 세 번째 데이터부터 마지막 데이터까지 출력하는 코드는 `print(menu[2:])`이다.

④ `print(menu[len(menu)])`를 실행하면 '핸드드립'이 출력된다.

⑤ 메뉴를 오름차순으로 정렬하는 코드는 `menu.sort(reverse=False)`이며, `reverse =False`를 생략해도 같은 결과가 나온다.

정답 및 해설

6.2

제어구조로 리스트 다루기

6.2.1 조건문으로 리스트 다루기

반복문에서도 리스트를 사용하지만, 조건문에서도 리스트를 사용할 수 있습니다. 이번에는 조건문에서 리스트를 어떻게 활용하는지 배워 보겠습니다.

음식을 좋아하는 순서대로 저장한 `food` 리스트를 만들고 음식 이름을 입력받아 해당 음식의 순위를 출력해 봅시다. 만약 입력받은 음식이 리스트에 없으면 순위에 없다고 출력합니다. 코드를 봅시다.

```
food = ['피자', '치킨', '스테이크', '된장찌개']
search = input('찾는 음식은? ')
if search in food:
    print(food.index(search) + 1, '위입니다.')
else:
    print(search, '은/는 순위에 없습니다.')
```

`food` 리스트를 만들고 음식 이름을 입력받아 `search` 변수에 저장합니다. `if-else` 조건문으로 해당 음식이 `food` 리스트에 있는 경우와 없는 경우를 나눕니다. 반복문에서 리스트에 원하는 데이터가 있는지 확인할 때 `in`을 사용했습니다. 조건문에서도 `in`을 사용해 `if search in food`로 작성합니다. 입력받은 음식이 리스트 `food`에 있으면 `index()`로 해당 음식의 인덱스를 구합니다. 인덱스는 0부터 시작하고 순위는 1위부터 시작합니다. 따라서 `index()`로 찾은 결과에 1을 더해야 순위가 제대로 나옵니다. 입력받은 음식이 리스트에 없다면 `else`로 가서 해당 음식이 순위에 없다고 출력합니다. 리스트에 있는 음식도 넣고, 없는 음식도 넣어서 실행해 보세요!

> `실행결과`
> 찾는 음식은? 치킨
> 2 위입니다.
> 찾는 음식은? 파스타
> 파스타 은/는 순위에 없습니다.

이번에는 영상 추천 프로그램을 만들어 보겠습니다. 유튜브에서 어떤 영상을 시청하고 나면 해당 영상과 관련한 영상을 추천하는 것을 본 적이 있을 겁니다. 또는 어떤 단어를 검색하면 해당 키워드와 연관된 영상을 추천하죠. 이런 추천 시스템은 매우 고도화되고 정교한 알고리즘으로 운영되지만, 우리는 프로그래밍을 처음 배우는 단계이므로 추천 알고리즘의 외형만 흉내내 봅시다.

우선 시청 내역을 보관하는 리스트가 하나 있어야 하니 `history`라는 이름으로 만듭니다. 그리고 `a`가 시청 내역에 있으면 `a`와 관련된 `a`' 영상을 추천하게 합니다. 예를 들어, 시청 내역에 '강아지'가 있으면 '고양이' 영상을 추천하는 겁니다. 여기에서는 '강아지'가 시청 내역에 있으면 '고양이' 영상을, '댄스'가 시청 내역에 있으면 '음악방송' 영상을 추천하는 코드를 작성해 봅시다.

```
history = ['먹방', '강아지', '요리', '댄스']
if '강아지' in history:
    print('강아지 영상을 좋아하시면 고양이 영상도 추천합니다!')
if '댄스' in history:
    print('댄스 영상을 좋아하시면 음악방송 영상도 추천합니다!')
```

```
실행결과
강아지 영상을 좋아하시면 고양이 영상도 추천합니다!
댄스 영상을 좋아하시면 음악방송 영상도 추천합니다!
```

`history` 리스트에 시청 내역을 간단히 4개만 저장했습니다. 시청 내역에 강아지가 있으면 고양이 영상을 추천하는 코드를 작성하고, 시청 내역에 댄스가 있으면 음악방송 영상을 추천하는 코드를 각각 작성합니다. 다른 요소에도 조건문을 활용해 추천 알고리즘을 추가할 수 있습니다. 직접 조건문을 추가해 보세요.

이번에는 새로운 영상을 시청했을 때 시청 내역에 추가하는 코드를 작성해 봅시다. 시청 내역을 저장하는 `history`를 빈 리스트로 만들고 시청한 영상 5개를 입력받아 `history`에 추가해 봅시다.

```
history = []
# 새로운 종류의 영상 시청
for i in range(5):
    history.append(input('시청한 영상은? '))
print('지금까지 시청한 영상은', history)
if '다이어트' in history:
    print('다이어트 영상을 좋아하시면 홈트레이닝 영상도 좋아하실 것 같아요!')
if '뉴스' in history:
    print('뉴스 영상을 좋아하시면 속보 영상도 추천합니다!')
```

`input()`으로 시청한 영상을 입력받고 `append()`로 리스트에 추가합니다. `input()`이 먼저 실행돼야 하므로 `append()` 안에 `input()`을 넣어 입력받은 값을 바로 `history`에 추가합니다. 이를 5번 반복하므로 반복문을 사용합니다. 그러고 나서 이전 코드처럼 추천 알고리즘을 작성합니다. 재미를 위해 여기서는 내용을 조금 바꿨습니다. 프로그램을 실행해 5개의 영상 주제를 입력해 봅시다.

```
실행결과
시청한 영상은? 뉴스
시청한 영상은? 가성비 맛집
시청한 영상은? 다이어트
시청한 영상은? 마스크 사용법
시청한 영상은? 스마트폰 후기
지금까지 시청한 영상은 ['뉴스', '가성비 맛집', '다이어트', '마스크 사용법', '스마트폰 후기']
다이어트 영상을 좋아하시면 홈트레이닝 영상도 좋아하실 것 같아요!
뉴스 영상을 좋아하시면 속보 영상도 추천합니다!
```

이처럼 실생활의 다양한 상황을 리스트로 표현할 수 있습니다.

6.2.2 중첩 제어구조로 리스트 다루기

이번에는 조건문과 반복문을 함께 사용해 봅시다. 한 반에 25명의 학생이 있습니다. 모든 학생의 점수를 리스트에 저장하고, 70점 이상인 학생의 학생 번호와 점수를 출력합니다. 점수 25개를 직접 넣기에는 너무 많으니 `random` 라이브러리를 활용해 0부터 100 사이의 무작위 정수를 리스트에 추가하겠습니다. 학생 번호는 리스트에 저장된 순서대로 1번부터 25번이라고 가정합니다.

먼저 `score`라는 빈 리스트를 만듭니다. 그런 다음 `score` 리스트에 무작위 정수를 25개 넣습니다. 무작위 정수는 `random.randint()`로 만들 수 있습니다. 이를 `score.append()`에 넣어 무작위 정수를 `score` 리스트에 추가하는 코드를 작성합니다. 그리고 이 코드를 반복문으로 25번 반복하면 리스트에 점수 25개가 채워집니다. 반복하는 동안 변하는 `score` 리스트를 확인할 수 있게 `print()` 명령어도 반복문 내부에 넣어 봅시다.

```
import random

score = []
for i in range(25):
    score.append(random.randint(0, 100))
    print(score)
```

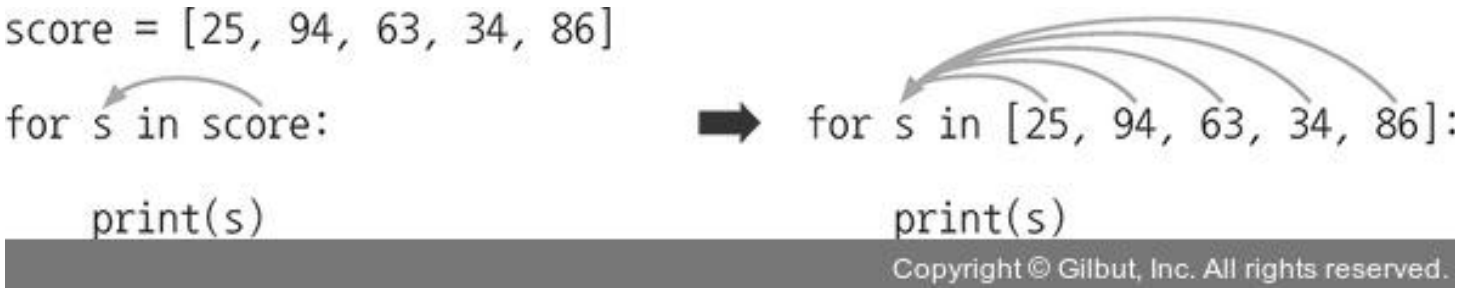
```
실행결과
[44]
[44, 33]
…
[44, 33, 27, 24, 4, 76, 45, 58, 84, 98, 60, 80, 19, 41, 94, 88, 83, 67, 44, 43, 70, 56, 69, 7
9, 14]
[44, 33, 27, 24, 4, 76, 45, 58, 84, 98, 60, 80, 19, 41, 94, 88, 83, 67, 44, 43, 70, 56, 69, 7
9, 14]
```

`score` 리스트가 변하는 과정이 보입니다(실행결과가 길어서 중간 결과는 생략했습니다). 무작위 정수를 넣으므로 이 책과 실행결과는 다를 수 있습니다.

`score` 리스트를 채웠으니 반에서 70점 이상인 학생의 번호와 점수를 출력해 봅시다. 먼저 리스트의 요소 값이 70 이상인 하나씩 확인해야 합니다. 리스트에 25개의 요소가 있으니 총 25번을 해야 합니다. 여기에서도 반복문을 사용합니다.

반복문으로 리스트의 요소에 접근하는 방법은 6.1 여러 데이터 한 번에 묶기에서 배웠듯이 두 가지입니다. 반복문 형식인 `for i in data`의 `data` 부분에 리스트를 그대로 넣어 요소에 직접 접근하는 방법과 `range()`를 사용해 인덱스로 접근하는 방법입니다. `data`에 어떤 것을 넣느냐에 따라 `i` 변수에 담기는 값이 달라집니다. 사실 반복문에서 다룬 부분이지만, 리스트를 배우기 전이었으므로 이번에 확실히 이해하고 넘어갑시다.

간단히 설명하기 위해 `score`에 저장된 값이 `[25, 94, 63, 34, 86]`이라고 가정합니다. 반복문에서 `data` 위치에 `score` 리스트를 넣으면 컴퓨터는 다음과 같이 해석합니다.



Copyright © Gilbut, Inc. All rights reserved.

`score`라고 리스트명을 적었지만, `score`는 `[25, 94, 63, 34, 86]`을 담고 있으므로 컴퓨터는 코드를 오른쪽 그림처럼 인식하죠. 따라서 `s` 변수에(인덱스가 아니므로 변수를 `i`가 아닌 `score`의 `s`로 설정) 25, 94, 63, 34, 86이 차례대로 저장됩니다. 가장 먼저 `s`에 25가 저장되면 `print(s)`가 실행되어 25가 출력되고, 다음으로 `s`에 94가 저장되면 94가 출력됩니다. 이처럼 `s`에 리스트의 요소가 직접 저장됩니다.

이번에는 `data` 위치에 리스트가 아닌 인덱스를 넣어 리스트를 다뤄 봅시다. 0부터 4까지의 인덱스 범위를 나타낸다면 `range(5)`를 사용합니다.



Copyright © Gilbut, Inc. All rights reserved.

`range(5)`는 `[0, 1, 2, 3, 4]`로 표현할 수 있습니다(실제로는 리스트 타입이 아니지만, 여기서는 설명하기 쉽게 데이터 묶음으로 표현했습니다). 따라서 `i` 변수에는 0부터 1씩 증가한 숫자가 차례대로 들어갑니다. 이때 `i` 변수는 `score` 리스트의 인덱스가 되어 `score` 리스트의 모든 요소를 출력할 수 있습니다.

그럼 두 방법 모두 코드로 작성해 보겠습니다. 이번에는 `print()`를 반복문 외부로 빼서 한 번만 실행하게 합니다.

```
import random

score = []
for i in range(25):
    score.append(random.randint(0, 100))
print(score)

print('리스트를 사용해 70점 이상인 점수 출력하기')
for s in score:
    if s >= 70:
        print(score.index(s) + 1, '번', s)

print('인덱스를 사용해 70점 이상인 점수 출력하기')
for i in range(len(score)):   # 또는 for i in range(25)
    if score[i] >= 70:
        print(i + 1, '번', score[i])
```

```
실행결과
[93, 37, 94, 38, 73, 75, 99, 40, 87, 93, 9, 76, 31, 40, 54, 31, 25, 77, 40, 46, 40, 39, 45, 9, 68]
리스트를 사용해 70점 이상인 점수 출력하기
1 번 93
3 번 94
…
인덱스를 사용해 70점 이상인 점수 출력하기
1 번 93
3 번 94
…
```

첫 번째 코드에서는 리스트를 그대로 반복문의 `data` 부분에 넣습니다. 점수가 `s` 변수에 담기고 `s`의 값이 70 이상일 때만 학생 번호와 점수를 출력합니다. 이때 학생 번호는 1부터 시작하므로 `s` 값의 인덱스인 `score.index(s)`에 1을 더해 출력하고, 점수는 `s` 변수의 값을 그대로 출력합니다.

두 번째 코드에서는 반복문의 `data` 부분에 연속된 숫자 범위를 나타내는 `range()`를 넣습니다. 또는 리스트 길이를 사용하면 `range(len(score))`가 됩니다. `score`에는 25개의 요소가 있으므로 `range(25)`로 작성해도 됩니다. 이때 `i` 변수는 리스트의 인덱스가 되어 `score[i]`가 70 이상일 때만 학생 번호와 점수를 출력하는 명령어가 실행됩니다. 여기서 학생 번호는 인덱스에 1을 더한 `i + 1`이 되고, 점수는 `score[i]`가 됩니다.

두 방법 모두 결과는 같습니다. 인덱스로 반복문을 작성하는 방법이 어렵다면 다음을 참고하세요.

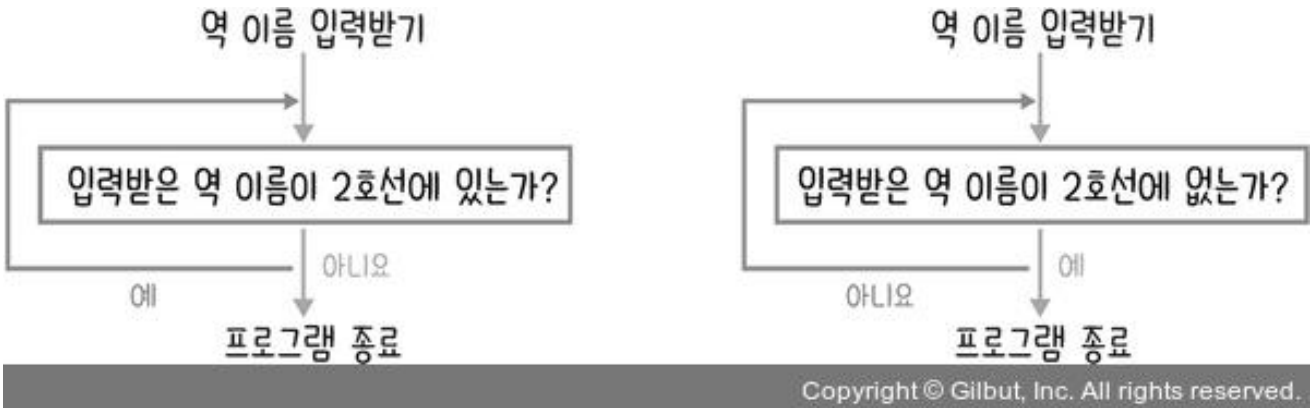


이처럼 조건문과 반복문에서 리스트를 자주 사용하니 확실하게 이해하고 넘어가기 바랍니다.

6.2.3 리스트로 간단한 프로그램 만들기

이번에는 익숙한 게임을 리스트를 사용해 프로그램으로 구현해 보겠습니다. 지하철 게임이라는 놀이가 있습니다. 술래가 호선을 지정하면 오른쪽으로 돌아가며 해당 호선에 있는 역을 차례대로 말합니다. 틀린 역을 말하거나 박자가 늦으면 벌칙을 받는 게임이죠. 예를 들어, 술래가 '2호선'을 외치면 2호선에 포함된 역만 말해야 합니다. 2호선이 아닌 다른 호선에 있는 역을 말하면 벌칙을 받습니다.

실제로는 모든 호선을 포함하지만, 여기서는 간단하게 서울 지하철 2호선을 대상으로 지하철 게임을 만들어 보겠습니다. 코드를 작성하기 전에 프로그램을 구상해 봅시다. 우선 역 이름을 입력받아야 합니다. 입력받은 역 이름이 2호선에 있으면 다음 역 이름을 입력받습니다. 입력받은 역 이름이 2호선에 없으면 게임을 멈춰야 합니다. 정리하면, 역 이름을 입력받는 것이 반복되고 입력받은 역 이름이 2호선이 아니라면 반복을 중단하는 것이죠. 이럴 때는 반복문과 반복문을 탈출하는 `break`를 사용하면 됩니다. 이해하기 쉽게 순서도로 표현해 보겠습니다.

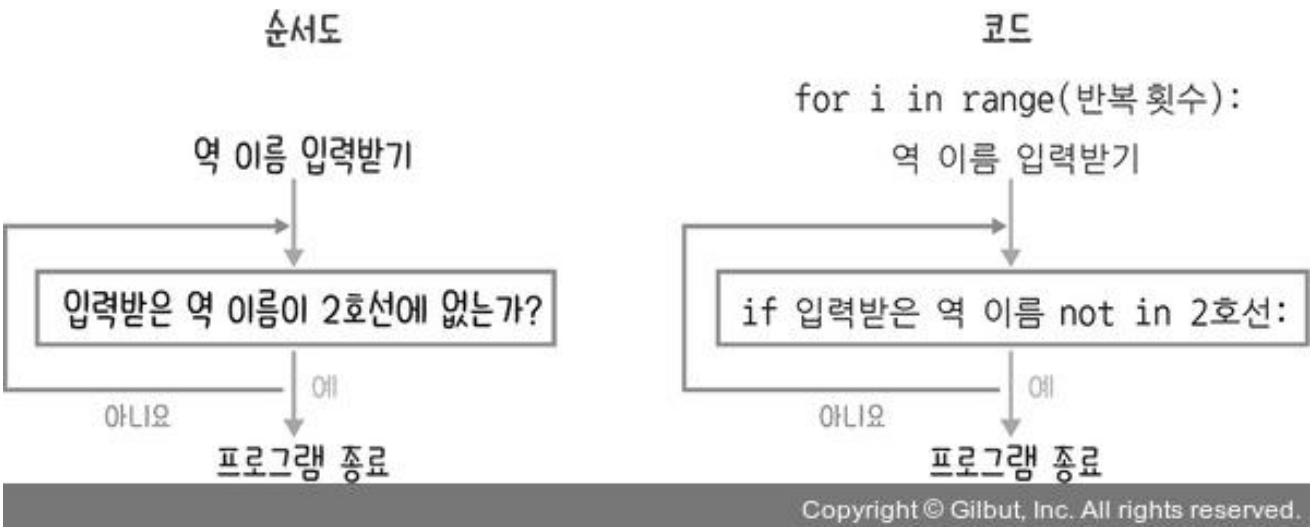


두 순서도는 동일하게 동작합니다. 왼쪽 순서도부터 봅시다. 입력받은 역이 2호선에 있으면 게임을 계속해야 하므로 '예' 화살표를 따라가서 다시 입력받는 명령어를 수행합니다. 이때 '예' 화살표는 반복문을 의미합니다. 입력받은 역이 2호선에 없으면 틀렸으므로 '아니요' 화살표를 따라가서 프로그램을 종료합니다. 눈치챘겠지만 두 갈래로 갈라지는 초록색 상자는 조건문을 의미합니다. '입력받은 역 이름이 2호선에 있는가?'가 조건이 되겠죠.

오른쪽 순서도는 왼쪽 순서도에서 조건문의 조건을 조금 변경했습니다. 조건이 '입력받은 역 이름이 2호선에 있는가?'에서 '입력받은 역 이름이 2호선에 없는가?'로 바뀌었습니다. 조건이 변경되면 '예'와 '아니요' 부분도 변경됩니다. 여기에서는 오른쪽 순서도를 기준으로 프로그램을 작성하겠습니다.

> 제제쌤의 조언
> 순서도는 원래 사용하는 모양이 따로 있지만, 여기서는 간단하게 표현했습니다. 순서도는 프로그램의 흐름을 도식화한 것이고, 요소들을 화살표로 연결해 순서를 나타낸다고 알면 됩니다.

컴퓨터는 입력받은 역 이름이 2호선에 없는지를 어떻게 판단할까요? 2호선에는 총 43개의 역이 있습니다(지선을 제외하고 본선만 센 숫자입니다). 43개 역 이름을 하나의 리스트로 묶으면 입력받은 역이 리스트에 없는지 `not in`으로 판단할 수 있습니다. 앞의 순서도에서 반복문과 조건문인 부분을 코드로 표현하면 다음과 같습니다.



그림을 참고해 2호선 지하철 게임을 작성해 보겠습니다. 먼저 2호선 43개 역 이름을 담은 리스트를 만듭니다. 리스트명은 line2로 하고 43개 역 이름을 리스트에 넣은 후 리스트 길이를 출력해 봅시다.

```
line2 = ['성수', '뚝섬', '한양대', '왕십리', '상왕십리', '신당', '동대문역사문화공원','을지로4가',
'을지로3가', '을지로입구', '시청', '충정로', '아현', '이대', '신촌', '홍대입구', '합정', '당산',
'영등포구청', '문래', '신도림', '대림', '구로디지털단지', '신대방', '신림', '봉천', '서울대입구',
'낙성대', '사당', '방배', '서초', '교대', '강남', '역삼', '선릉', '삼성', '종합운동장', '잠실새
내', '잠실', '잠실나루', '강변', '구의', '건대입구']
print(len(line2))
```

```
실행결과
4
```

> 제제쌤의 조언
> 43개 역을 직접 입력하기 어려우면 이 책의 소스 코드에서 해당 부분만 복사해서 사용해도 됩니다. 또는 간략하게 몇 개의 역만 리스트로 작성해도 됩니다. 실습을 위한 프로그램이므로 완벽하게 만들 필요는 없습니다.

`line2` 리스트의 길이가 43이라고 나옵니다. 제대로 입력됐네요.

순서도를 보면 반복문 안에서 역 이름을 입력받고 해당 역이 2호선 안에 없는지를 확인합니다. 반복문을 사용하려면 반복 횟수를 정해야 하는데 여기서는 10회를 반복한다고 가정하겠습니다. 역 이름을 입력받아야 하므로 반복문 내부에 `input()`이 있어야 합니다. 그리고 `input()` 값이 `line2` 리스트에 없으면 프로그램을 종료합니다.

코드로 작성하면 다음과 같습니다.

```
print('지하철~ ' * 4)
# 10번 반복한다고 가정
for i in range(10):
    station = input()
    if station not in line2:
        print('땡! 해당 역은 2호선이 아닙니다!')
        break
```

`input()`으로 입력받은 값을 `station` 변수에 저장하고 `station` 값이 `line2` 리스트에 없으면 반복을 멈추도록 `break`를 넣습니다. 입력이 틀리면 프로그램을 종료하기 전에 입력이 틀렸다는 메시지도 출력합니다. 프로그램을 실행해 다양한 값을 넣어 봅시다.

```
실행결과
지하철~ 지하철~ 지하철~ 지하철~
강남
합정
역삼
낙성대
홍대
땡! 해당 역은 2호선이 아닙니다!
```

단순하긴 하지만 실제 놀이를 파이썬으로 작성해 봤습니다. 본인이 아는 다른 놀이로도 한번 작성해 보세요.

1분 퀴즈

3 다음 코드에 대한 설명으로 옳지 않은 것을 고르세요.

```
import random

score = []
A = []
B = []
C = []

for i in range(30):
    score.append(random.randint(0, 100))
for i in range(len(score)):
    if score[i] > 80:
        A.append(score[i])
    elif score[i] > 40:
        B.append(score[i])
    else:
        C.append(score[i])
```

① `score`에는 총 30개의 무작위 정수가 저장된다.

② `score`에서 80을 초과하는 숫자는 `A` 리스트에 저장된다.

③ `score`에 숫자 40이 있다면 `C` 리스트에 저장된다.

④ `A`, `B`, `C` 리스트 중 하나에 저장된 숫자는 `score` 리스트에서 삭제된다.

⑤ 두 번째 반복문은 다음과 같이 바꿔도 결과가 동일하다.

```
for s in score:
    if s > 80:
        A.append(s)
    elif score[i] > 40:
        B.append(s)
    else:
        C.append(s)
```

정답 및 해설

6.3

여러 개의 리스트 묶기

6.3.1 이차원 리스트란

지금까지 여러 데이터를 묶어 하나의 리스트로 만들었습니다. 그런데 이런 리스트가 여러 개 있을 때 이를 다시 리스트로 묶을 수 있습니다. 리스트 안에 리스트가 있는 것이죠. 예를 들면 다음과 같은 형태입니다.

```
abc = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

`abc` 리스트에 3개의 요소가 들어 있습니다. `abc` 리스트의 각 요소 또한 리스트입니다. 이처럼 리스트 안에 리스트가 들어 있는 형태를 이차원 리스트라고 합니다. 따라서 앞서 배운 리스트는 일차원 리스트가 됩니다.

TIP 리스트를 표현하는 대괄호가 많으므로 개수에 유의하세요. 빨간색은 전체 리스트를 의미하고 파란색은 abc 리스트의 요소가 되는 리스트를 의미합니다.

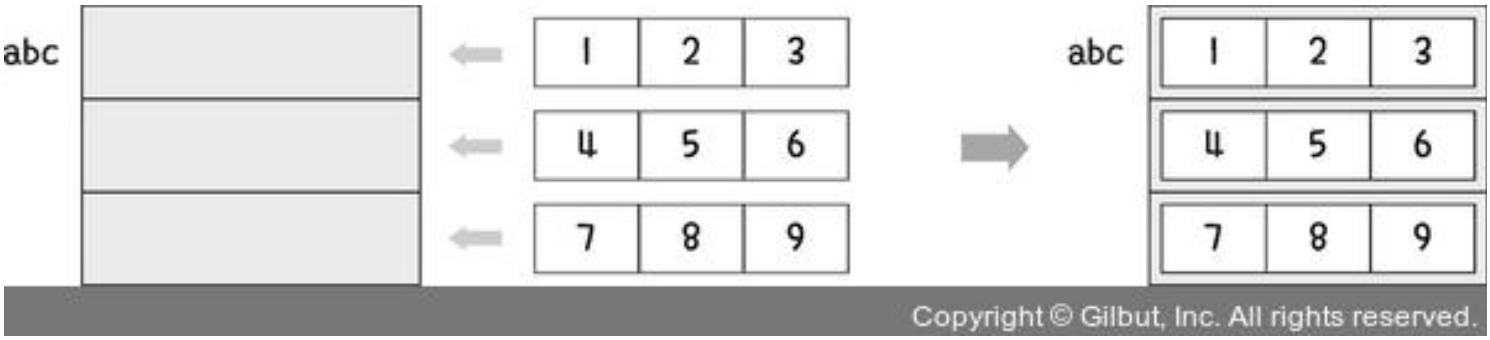
```
abc = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

이차원 리스트를 앞의 코드처럼 한 줄로 작성하면 이차원 리스트인지 한눈에 파악하기 어렵기 때문에 다음처럼 줄바꿈하기도 합니다.

```
abc = [[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]]
```

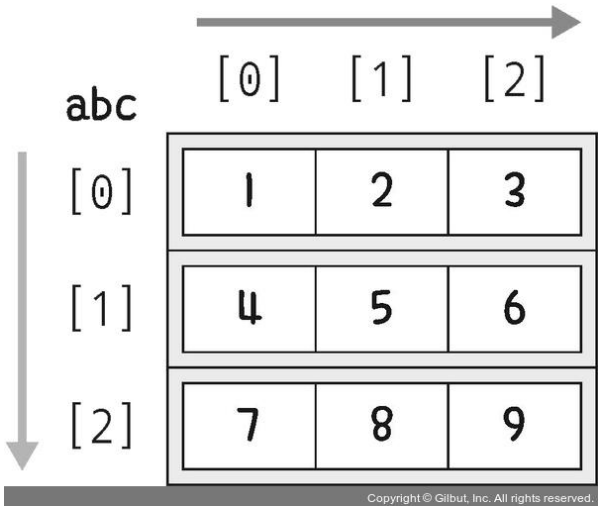
두 코드는 같은 코드입니다. 줄바꿈이 있고 없고의 차이일 뿐이니 편한 방식으로 코드를 작성하면 됩니다.

코드를 실행하면 다음 그림처럼 리스트가 만들어집니다. 일차원 리스트는 한 행만 있는 일차원 데이터였다면 이차원 리스트는 행과 열이 있는 이차원 데이터입니다.



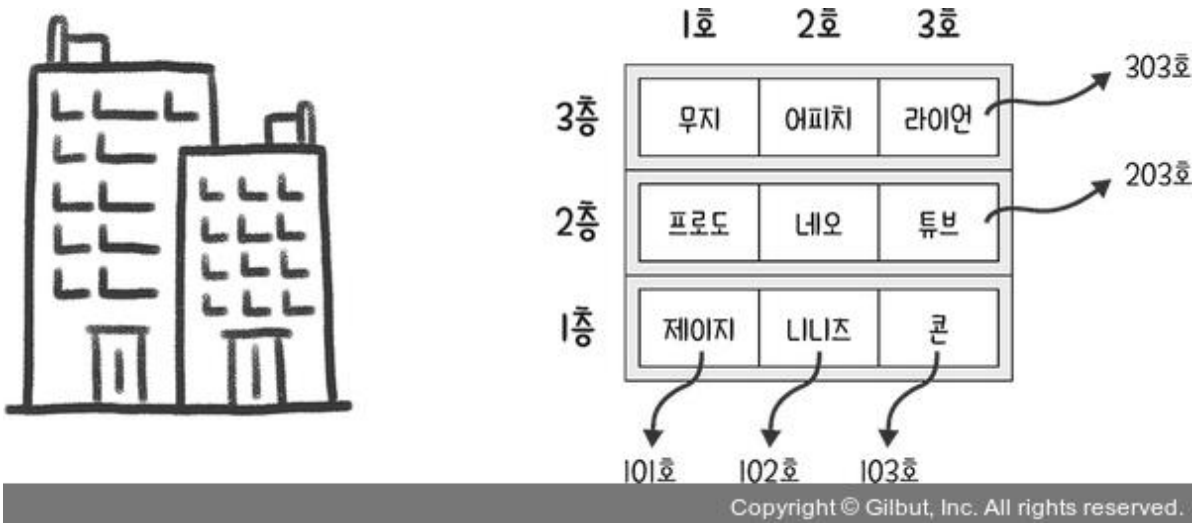
▲ 그림 6-1 이차원 리스트의 구조

그럼 이차원 리스트의 데이터에는 어떻게 접근할까요? 일차원 리스트는 인덱스 1개로 개별 데이터를 불러올 수 있었는데, 이차원 리스트에서는 인덱스가 2개(행과 열)입니다.



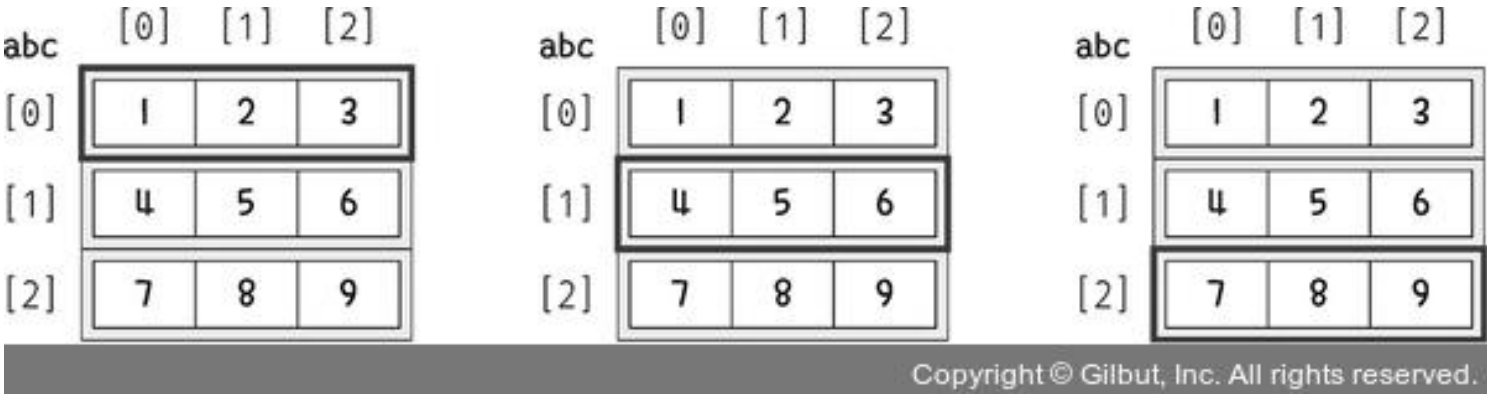
▲ 그림 6-2 이차원 리스트의 인덱스

인덱스가 2개이기 때문에 두 인덱스를 함께 사용해 데이터에 접근합니다. 인덱스가 2개이니 어떻게 읽어야 할지 모르겠죠? 쉬운 예를 들어 봅시다. 아파트에는 여러 층이 있고, 각 층에는 여러 집이 있습니다. 층은 1층, 2층, 3층으로 구분하고, 한 층의 집은 1호, 2호, 3호로 구분합니다. 그리고 이 둘을 합쳐서 1층의 3호는 103호로 부릅니다. 6층의 12호면 612호가 되겠죠. 규칙을 파악했나요? 층과 호를 결합하는데, 층이 먼저 오고 그 뒤에 호가 붙는 방식으로 아파트의 호수를 붙입니다.



▲ 그림 6-3 아파트 호수 구조

이차원 리스트도 동일합니다. 층에 해당하는 행 인덱스를 먼저 쓰고 바로 뒤에 호에 해당하는 열 인덱스를 써서 개별 데이터를 가리킵니다. 다만, 아파트와 다른 부분은 행 인덱스가 위에서 아래로 내려가는 형태라는 점입니다. 리스트는 위에서 시작하므로 가장 윗줄이 행 인덱스 `[0]`이 됩니다.



▲ 그림 6-4 이차원 리스트의 행 인덱스

이차원 리스트를 구성하는 요소는 일차원 리스트입니다. 이차원 리스트의 각 요소를 출력해 봅시다. 일차원 리스트처럼 리스트명 옆에 인덱스를 붙이면 됩니다.

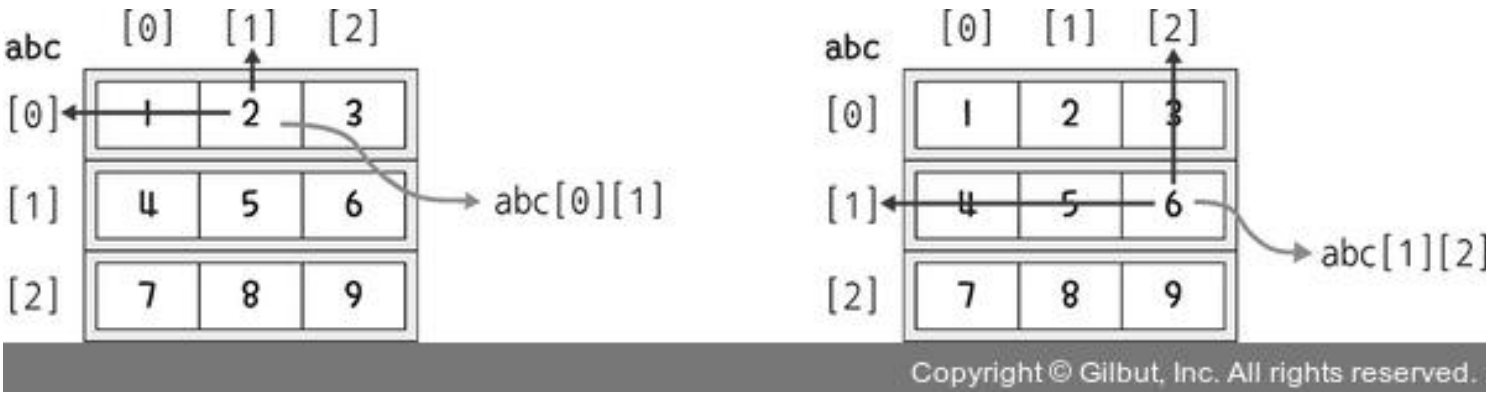
```
abc = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(abc[0])
print(abc[1])
print(abc[2])
```

실행결과
```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

이차원 리스트의 요소는 일차원 리스트이므로 `abc[i]` 형태를 출력하면 각각의 일차원 리스트가 출력됩니다. 이렇게 이차원 리스트명 옆에 붙인 인덱스가 행 인덱스입니다.

행 인덱스와 열 인덱스를 결합해 보겠습니다. 예로 든 아파트 형태와 읽는 방법이 동일합니다. 숫자 2가 들어 있는 칸을 봅시다. 행 인덱스가 `[0]`이고 열 인덱스가 `[1]`이므로 `abc[0][1]`로 나타냅니다. 숫자 6이 들어 있는 칸도 확인해 봅시다. 행 인덱스가 `[1]`이고 열 인덱스가 `[2]`이니 `abc[1][2]`입니다.

> 제제쌤의 조언
> 행과 열은 다소 헷갈릴 수 있습니다. 그럴 때 행렬을 떠올려 보세요. 이때 가로가 행이고, 세로가 열입니다. 행과 열은 가로와 세로를 한 글자로 표현한 용어입니다. 일반적으로 '가로 세로'라고 하지 '세로 가로'라고 하지 않죠? 가로(행) 세로(열)로 인지하면 덜 헷갈릴 거예요.

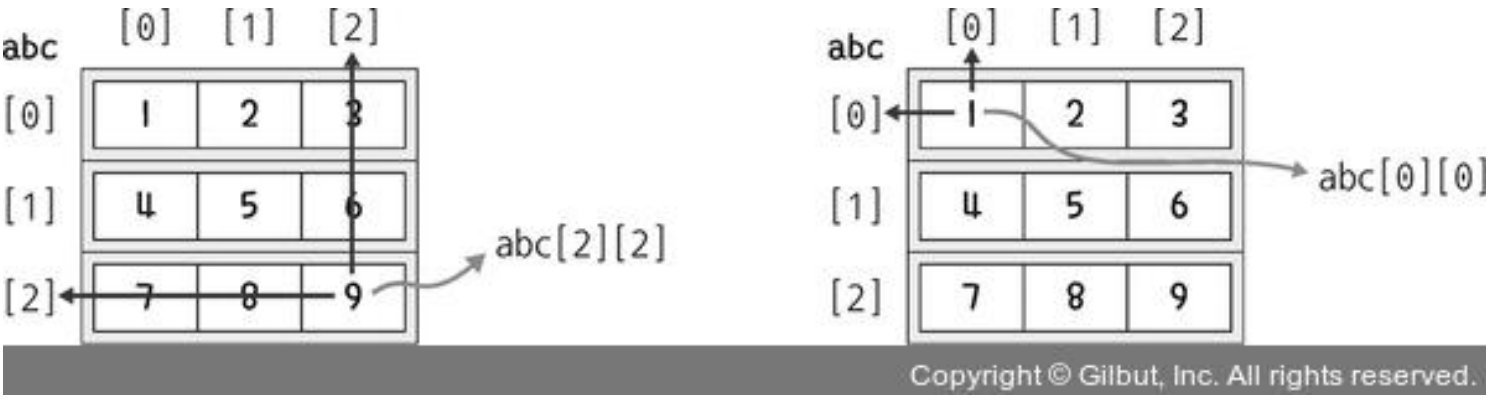


▲ 그림 6-5 이차원 리스트에서 숫자 2와 6의 인덱스

```
abc = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(abc[0][1])
print(abc[1][2])
```

```
실행결과
2
6
```

그럼 9는 어떻게 출력할까요? 행 인덱스가 `[2]`, 열 인덱스가 `[2]`이므로 `print(abc[2][2])`를 실행하면 됩니다. 1도 출력해 볼까요? 행 인덱스가 `[0]`, 열 인덱스가 `[0]`이므로 `print(abc[0][0])`을 실행하면 됩니다.



▲ 그림 6-6 이차원 리스트에서 숫자 9와 1의 인덱스

이처럼 2개의 인덱스를 사용해 이차원 리스트의 데이터에 쉽게 접근할 수 있습니다. 행 인덱스와 열 인덱스를 확인하고 행과 열 순서로 결합해 작성합니다.

NOTE 이차원 리스트 안의 일차원 리스트 길이

이차원 리스트에는 다음 코드처럼 각기 다른 길이의 일차원 리스트도 들어갈 수 있습니다.

```
abc = [[1, 2],
       [3, 4, 5, 6],
       [7, 8, 9],
       [10, 11]]
print(abc)
```

실행결과

```
[[1, 2], [3, 4, 5, 6], [7, 8, 9], [10, 11]]
```

`abc[0]`, `abc[1]`, `abc[2]`, `abc[3]`은 길이가 각각 다른 일차원 리스트지만 이들을 합쳐 이차원 리스트를 만들 수 있습니다. 다만, 이 경우에는 행 인덱스마다 열 인덱스가 다르기 때문에 코딩할 때 열 인덱스의 범위를 알고 사용해야 합니다. 그렇지 않으면 `IndexError`가 발생할 수 있어요.

6.3.2 반복문으로 이차원 리스트 다루기

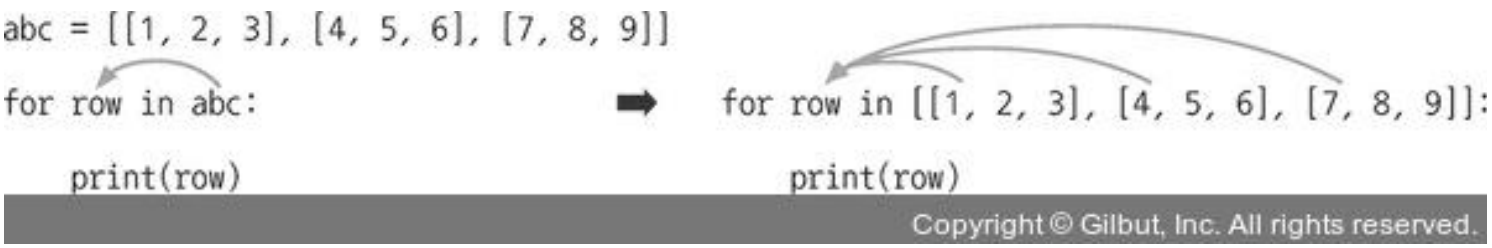
이번에는 반복문에 이차원 리스트를 직접 넣어 보겠습니다. 반복문으로 이차원 리스트를 다룰 때에는 일반적으로 `row`라는 변수를 사용합니다. `row`는 우리말로 열(줄)을 뜻합니다. 간단하게 한 열, 한 줄이라는 의미인데 왜 이차원 리스트의 반복문의 변수로 `row`를 사용할까요? 다음 코드를 실행해 보고 그 이유를 생각해 봅시다.

```
for row in abc:
    print(row)
```

```
실행결과
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

새로운 변수 `row`가 나왔지만 형식은 그대로입니다. `for i in data`에서 `data`에 리스트를 넣으면 리스트에 들어 있는 값 자체가 차례대로 `i` 변수에 들어갔습니다. 여기서도 같습니다.

이차원 리스트의 반복문은 다음 그림처럼 진행됩니다. 이차원 리스트의 요소는 일차원 리스트입니다. 이차원 리스트 `abc`의 첫 번째 요소(일차원 리스트)부터 마지막 요소(일차원 리스트)까지 순서대로 `row` 변수에 담깁니다.



Copyright © Gilbut, Inc. All rights reserved.

반복이 시작되면 `abc` 리스트의 첫 번째 데이터인 `[1, 2, 3]`이 `row` 변수에 들어가고 `print(row)`를 실행합니다. 두 번째 반복에서 두 번째 데이터인 `[4, 5, 6]`이, 마지막 반복에서 `[7, 8, 9]`가 `row`에 들어가고 `print(row)`를 실행합니다. 반복할 때마다 이차원 리스트의 한 행이 `row` 변수에 담깁니다.

▼ 표 6-2 반복문의 실행과정

반복 회차	row에 담기는 요소	반복문 내부
1	[1, 2, 3]	
2	[4, 5, 6]	
3	[7, 8, 9]	

이차원 리스트는 일차원 리스트 여러 개로 구성되기 때문에 반복문의 변수에는 이차원 리스트의 요소인 일차원 리스트가 순서대로 저장됩니다. 그런데 일차원 리스트가 눈으로 보기에는 옆으로 늘어진 한 줄처럼 보이기 때문에 변수 `row`를 사용하는 것이죠.

한 단계 더 들어가 보겠습니다. `row`에는 일차원 리스트가 담깁니다. 따라서 `row`에도 인덱스를 적용할 수 있습니다. `row[0]`, `row[1]`, `row[2]`처럼요. `row`에 첫 번째 리스트 `[1, 2, 3]`이 들어 있을 때 `row[0]`은 1, `row[1]`은 2, `row[2]`는 3입니다. `row`에 두 번째 리스트 `[4, 5, 6]`이 들어 있으면 `row[0]`, `row[1]`, `row[2]`는 각각 4, 5, 6이죠. 다음 코드를 실행해 봅시다.

```
for row in abc:
    print(row[0])
```

```
실행결과
1
4
7
```

`row`에 `[1, 2, 3]`이 들어 있을 때 `row[0]`은 1, `[4, 5, 6]`이 있을 때는 4, `[7, 8, 9]`가 있을 때는 7이 출력됩니다. 즉, 이차원 리스트 `abc`의 첫 번째 열이 출력됩니다.

이번에는 마지막 열만 출력해 보겠습니다. `row`의 마지막이므로 인덱스를 `row[-1]`로 작성합니다.

```
for row in abc:
    print(row[-1])
```

```
실행결과
3
6
9
```

▼ 표 6-3 반복문의 실행과정

반복 회차	row에 담기는 요소	row[0]	row[-1]
1	[1, 2, 3]	1	3
2	[4, 5, 6]	4	6
3	[7, 8, 9]	7	9

이처럼 반복문의 `data` 위치에는 이차원 리스트도 넣을 수 있습니다.

6.3.3 이차원 리스트로 간단한 프로그램 만들기

그럼 이차원 리스트는 언제, 어떻게 쓰일까요? 사실 일상생활에는 이차원 리스트가 정말 많습니다. 표로 표현되는 것은 대부분 이차원 리스트로 작성할 수 있습니다. 표는 데이터가 잘 정리되고 구조화된 형태입니다. 우리는 하루에도 몇 번씩 표를 봅니다. 뉴스에서 흔하게 접할 수 있고 게임에서도 플레이어들의 랭킹 순위를 표 형태로 제공합니다. 다음 제시된 카드 내역도 표입니다.



2020.03.04 18:56	주식회사 카카오페이	1,000원
2020.03.03 14:54	네이버페이	51,100원
2020.03.02 12:13	스타벅스코리아	10,000원
2020.03.01 15:49	11번가	27,500원
2020.02.26 11:40	주식회사 카카오페이	32,500원
2020.02.26 11:38	주식회사 카카오페이	32,500원
2020.02.22 15:00	G마켓	81,500원
2020.02.22 15:15	네이버페이	131,000원

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 6-7 표로 된 카드 내역

표는 행과 열이 있으므로 이차원 리스트로 만들 수 있습니다. 카드 내역을 이차원 리스트로 만들면 다음과 같습니다.

```
card = [['2020/03/04', '주식회사 카카오페이', 1000],
        ['2020/03/03', '네이버페이', 51100],
        ['2020/03/02', '스타벅스코리아', 10000],
        ['2020/03/01', '11번가', 27500],
        ['2020/02/26', '주식회사 카카오페이', 32500],
        ['2020/02/26', '주식회사 카카오페이', 32500],
        ['2020/02/22', 'G마켓', 81500],
        ['2020/02/22', '네이버페이',131000]]
```



▲ 그림 6-8 이차원 리스트로 구현한 카드 내역

카드 내역에 8건이 있으므로 총 8개의 행이 만들어집니다. 각 행은 [이용일시, 가맹점명, 이용금액]이 있는 일차원 리스트가 됩니다. 그리고 일차원 리스트는 이차원 리스트 `card`의 각 요소로 저장됩니다. 따라서 `card` 리스트에 인덱스를 붙이면 원하는 카드 내역을 알 수 있습니다.

```
print(card[0])
print(card[3])
```

```
실행결과
['2020/03/04', '주식회사  카카오페이', 1000]
['2020/03/01', '11번가', 27500]
```

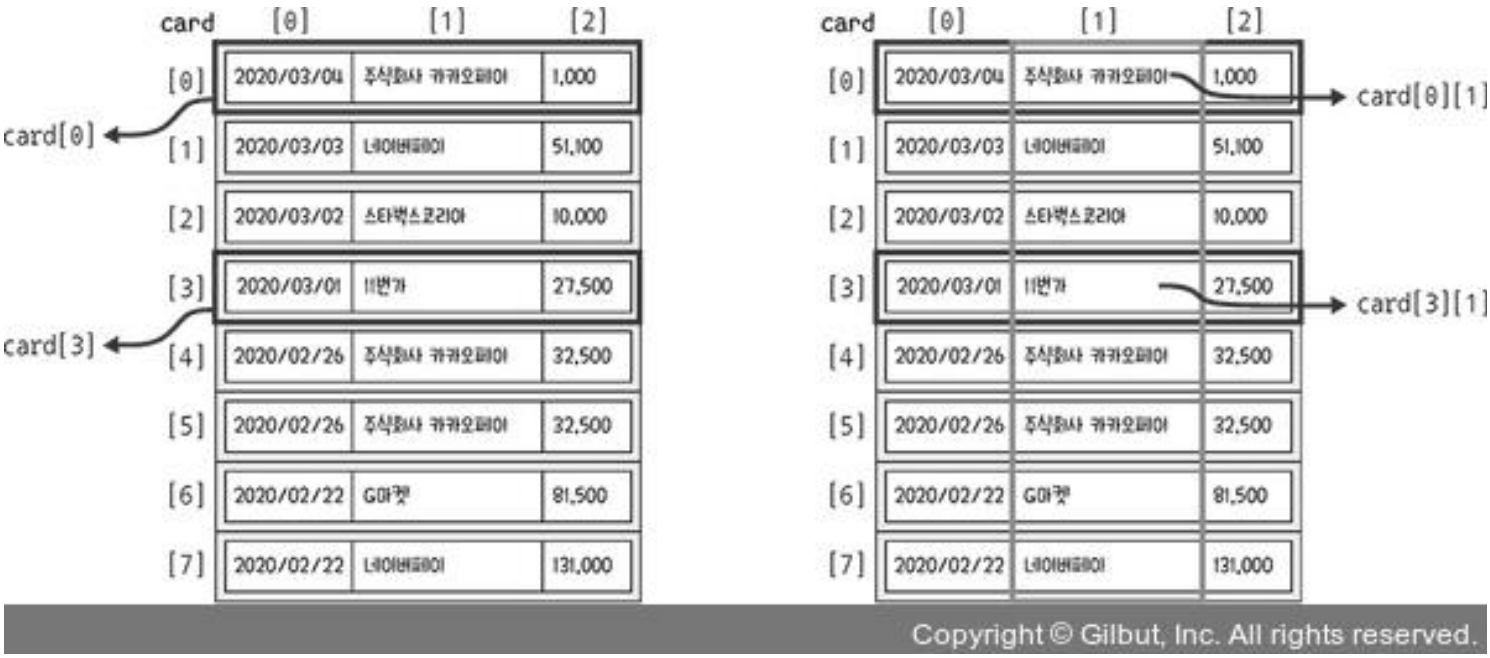
`card[0]`을 출력하니 첫 번째 행이, `card[3]`을 출력하니 네 번째 행이 리스트로 출력됩니다.

그럼 첫 번째 카드 내역과 네 번째 카드 내역의 가맹점명만 출력해 봅시다.

```
print(card[0][1])
print(card[3][1])
```

```
실행결과
주식회사  카카오페이
11번가
```

가맹점명은 각 행의 두 번째 열에 있으므로 열 인덱스는 `[1]`이 됩니다. 따라서 첫 번째 카드 내역과 네 번째 카드 내역의 가맹점명은 각각 `card[0][1]`과 `card[3][1]`로 구할 수 있습니다.



▲ 그림 6-9 첫 번째와 네 번째 카드 내역의 가맹점명

이번에는 마지막 행의 가맹점명과 이용금액을 동시에 출력해 봅시다.

```
print(card[-1][1], card[-1][-1])
```

```
실행결과
네이버페이 131000
```

마지막 행의 인덱스는 `[-1]`이죠. 따라서 `card[-1]`은 `['2020/02/22', 네이버페이', 131000]`입니다. 리스트에서 가맹점명과 이용금액은 두 번째 요소와 마지막 요소이므로 `card[-1][1]`, `card[-1][-1]`로 작성할 수 있습니다. 물론 `card[-1][1]`, `card[-1][2]`로 작성해도 결과는 같습니다.

다른 예를 들어 볼까요? 스포츠 경기에서 순위를 나타낼 때도 표 형태를 볼 수 있습니다. 다음은 2016년 브라질에서 열린 제31회 리우데자네이루 올림픽의 순위를 나타낸 표입니다.

▼ 표 6-4 올림픽 순위[1]

순위	국가	금메달	은메달	동메달	합계
1	미국	46	37	38	121
2	영국	27	23	17	67
3	중국	26	18	26	70
4	러시아	19	17	19	55
5	독일	17	10	15	42
6	일본	12	8	21	41
7	프랑스	10	18	14	42
8	대한민국	9	3	9	21

1위부터 8위까지 순위로 이차원 리스트를 만들어 보겠습니다.

```
olympic =[[1, '미국', 46, 37, 38, 121],
          [2, '영국', 27, 23, 17, 67],
          [3, '중국', 26, 18, 26, 70],
          [4, '러시아', 19, 18, 19, 55],
          [5, '독일', 17, 10, 15, 42],
          [6, '일본', 12, 8, 21, 41],
          [7, '프랑스', 10, 18, 14, 42],
          [8, '대한민국', 9, 3, 9, 21]]
```

1 https://ko.wikipedia.org/wiki/2016년_하계_올림픽_메달_집계

`olympic` 리스트에 1위부터 8위까지의 순위와 국가명, 금은동 메달 개수를 넣었습니다. 각 행에는 [순위, 국가명, 금메달 개수, 은메달 개수, 동메달 개수, 메달 합계]가 일차원 리스트 형태로 들어갑니다. 1위와 5위의 국가명과 금메달 개수를 한번 출력해 봅시다.

```
print(olympic[0][1], '금메달 개수:', olympic[0][2])
print(olympic[4][1], '금메달 개수:', olympic[4][2])
```

```
실행결과
미국 금메달 개수: 46
독일 금메달 개수: 17
```

1위는 행 인덱스 `[0]`에, 5위는 행 인덱스 `[4]`에 있습니다. 국가명은 열 인덱스로 `[1]`, 금메달 개수는 열 인덱스 `[2]`에 있고요. 따라서 이를 조합하고 중간에 적절한 출력 메시지를 추가하면 원하는 결과를 얻을 수 있습니다.

이차원 리스트 슬라이싱하기

이번에는 미국과 독일의 금은동 메달 개수를 한 번에 출력해 봅시다. 금메달 개수는 열 인덱스 `[2]`에, 은메달 개수는 열 인덱스 `[3]`에, 동메달 개수는 열 인덱스 `[4]`에 있습니다. 열 인덱스 `[2]`부터 `[4]`까지입니다. 이를 슬라이싱 형식으로 표현하면 `[2:5]`입니다. 따라서 미국이 들어 있는 행 인덱스 `[0]`에 열 인덱스 `[2:5]`를 붙여 출력하면 됩니다. 독일도 마찬가지로 행 인덱스 `[4]`에 행 인덱스 `[2:5]`를 붙이면 됩니다. 코드로 확인해 봅시다.

```
print(olympic[0][1], '금은동 메달 개수:', olympic[0][2:5])
print(olympic[4][1], '금은동 메달 개수:', olympic[4][2:5])
```

```
실행결과
미국 금은동 메달 개수: [46, 37, 38]
독일 금은동 메달 개수: [17, 10, 15]
```

금은동 메달 개수가 리스트로 나왔습니다. 이처럼 이차원 리스트에서도 슬라이싱할 수 있습니다. 열 인덱스에 슬라이싱을 적용하면 한 행 안에서 슬라이싱하게 됩니다.

그럼 행 인덱스에도 슬라이싱을 적용할 수 있을까요? 물론 가능합니다! 다음 코드를 실행해 보세요.

```
print(olympic[:4])
print(olympic[:4][1])
```

```
실행결과
[[1, '미국', 46, 37, 38, 121], [2, '영국', 27, 23, 17, 67], [3, '중국', 26, 18, 26, 70], [4,
'러시아', 19, 18, 19, 55]]
[2, '영국', 27, 23, 17, 67]
```

`olympic[:4]`는 `olympic` 리스트의 행 인덱스 `[0]`부터 `[3]`까지 슬라이싱합니다. 그래서 결과로 4개의 리스트가 나옵니다. 리스트 안에 4개의 리스트가 들어 있으므로 이 역시 또 다른 이차원 리스트입니다. 다음 그림을 참고하세요.



▲ 그림 6-10 이차원 리스트의 슬라이싱 결과

`olympic[:4]`가 이차원 리스트이므로 `olympic[:4][1]`을 실행하면 행 인덱스 `[1]`에 있는 일차원 리스트 `[2, '영국', 27, 23, 17, 67]`이 출력됩니다.

앞에서 실생활에서 접할 수 있는 표를 이차원 리스트로 표현해 보고 원하는 부분만 골라 출력해 보았습니다. 이차원 리스트를 처음 접하면 다소 어려울 수 있으나 사실 개념은 간단합니다. 흩어져 있는 데이터를 한눈에 보기 좋게 정리한 표를 컴퓨터가 인식하는 방식이 바로 이차원 리스트인 셈입니다.

앞에서 배운 내용을 바탕으로 조금 더 적극적으로 이차원 리스트를 다뤄 보겠습니다.

앞에서 2016년 올림픽의 메달 순위를 이차원 리스트로 만들고, 행 인덱스와 열 인덱스로 슬라이싱해 보았습니다. 이번에는 `olympic` 리스트에서 순위와 국가명만 골라 출력해 봅시다. 먼저 행 인덱스 `[0]`일 때 열 인덱스 `[0:2]`를 슬라이싱하면 되겠죠? 다음으로 행 인덱스 `[1]`일 때 `[0:2]`를 슬라이싱해 출력합니다. 이렇게 모든 행 인덱스에서 열 인덱스 `[0:2]`를 슬라이싱해 출력하면 됩니다. 반복되는 부분이 보이나요? 코드를 작성해 봅시다.

```
for row in olympic:
    print(row[:2])
```

실행결과

```
[1, '미국']
[2, '영국']
[3, '중국']
[4, '러시아']
[5, '독일']
[6, '일본']
[7, '프랑스']
[8, '대한민국']
```

반복문으로 이차원 리스트의 모든 행(일차원 리스트)을 차례대로 `row` 변수에 넣은 후, 반복문 안에서 일차원 리스트의 처음 두 요소를 `[:2]`로 선택하면 됩니다.

중첩 제어구조로 이차원 리스트 다루기

이번에는 금메달보다 은메달이 더 많은 나라를 찾아보겠습니다. 일차원 리스트에서도 각 요소에 접근하려면 반복문을 사용해야 했습니다. 이와 마찬가지로 이차원 리스트 `olympic`에서 금메달보다 은메달이 많은 나라를 찾으려면 각 행에 접근해서 '금메달 개수보다 은메달 개수가 많은가?'를 일일이 확인해야 합니다. 앞서 반복문으로 이차원 리스트를 다루는 방법을 배웠으니 이를 바탕으로 프로그램을 작성해 봅시다.

```
print('금메달보다 은메달이 더 많은 나라')
for row in olympic:
    if row[2] < row[3]:
        print(row[1])
```

```
실행결과
금메달보다 은메달이 더 많은 나라
프랑스
```

반복문을 한 번 실행할 때마다 일차원 리스트 하나가 `row`에 들어갑니다. 첫 번째 실행에서는 `row`에 `[1, '미국', 46, 37, 38, 121]`이 저장됩니다. 그리고 반복문 내부에서 조건문으로 금메달 개수 `row[2]`와 은메달 개수 `row[3]`을 비교합니다. `row[3]`이 큰 리스트가 있으면 해당 리스트의 `row[1]` 값을 출력합니다. 금메달보다 은메달 개수가 많은 나라는 프랑스뿐이네요.

▼ 표 6-5 중첩 제어구조 실행 과정

반복 회차	row에 저장되는 데이터	row[2]와 row[3]	조건문 충족 여부 (if row[2] < row[3])
첫 번째	`[1, '미국', 46, 37, 38, 121]`	`row[2]`는 46, `row[3]`은 37	충족함 `print(row[1])` 실행 안 함
두 번째	`[2, '영국', 27, 23, 17, 67]`	`row[2]`는 27, `row[3]`은 23	충족하지 않음 `print(row[1])` 실행 안 함
		...	
일곱 번째	`[7, '프랑스', 10, 18, 14, 42]`	`row[2]`는 10, `row[3]`은 18	충족함 `print(row[1])` 실행
마지막	`[8, '대한민국', 9, 3, 9, 21]`	`row[2]`는 9, `row[3]`은 3	충족하지 않음 `print(row[1])` 실행 안 함

이번에는 앞의 코드를 수정해 금메달보다 동메달이 많은 나라를 출력해 봅시다.

```
print('금메달보다 동메달이 더 많은 나라')
for row in olympic:
    if row[2] < row[4]:
        print(row[1])
```

```
실행결과
금메달보다 동메달이 더 많은 나라
일본
프랑스
```

이전 코드에서 은메달 개수인 `row[3]`을 동메달 개수인 `row[4]`로 변경하기만 하면 됩니다. 결과는 일본과 프랑스입니다. 프랑스는 금메달보다 은메달, 동메달이 더 많군요.

이번에는 1위부터 8위까지 모든 국가가 획득한 금메달 개수의 합을 구해 봅시다.

```
total_gold = 0
for row in olympic:
    total_gold += row[2]
print(total_gold)
```

```
실행결과
166
```

1위부터 8위까지 금메달의 총 개수를 저장하는 변수로 `total_gold`를 만들었습니다. `total_gold`를 만들 때는 아직 금메달 개수를 더한 상태가 아니므로 초깃값 0을 넣어 줍니다. 그런 다음 모든 일차원 리스트(이차원 리스트의 행)를 차례대로 `row`에 넣는 반복문을 작성합니다. 금메달 개수는 `row[2]`에 있으므로 일차원 리스트의 `row[2]`를 `total_gold`에 모두 더하면 모든 나라의 금메달 개수가 구해집니다. 최종 값은 166이 나옵니다. 이 결과가 맞는지 확인하고 싶으면 직접 더해 보세요.

이번에는 나라별 총 메달 개수 중 금메달이 차지하는 비율이 40%가 넘는 나라를 구해 보겠습니다. 총 메달 개수는 마지막 열 인덱스 즉, `[-1]`에 있고, 금메달 개수는 열 인덱스 `[2]`에 있습니다. 따라서 금메달이 총 매달 개수에서 차지하는 비율은 `row[2] / row[-1] * 100`으로 구할 수 있습니다.

```
for row in olympic:
    gold = row[2] / row[-1] * 100
    if gold > 40:
        print(row[1], gold, '%')
```

```
실행결과
영국 40.298507462686565 %
독일 40.476190476190474 %
대한민국 42.857142857142854 %
```

먼저 행마다 금메달 비율을 구해 `gold` 변수에 저장합니다. 그리고 `gold` 변수의 값이 40을 초과하면 해당 국가명과 비율을 출력합니다.

이처럼 실생활의 데이터를 이차원 리스트로 변경하고 반복문과 조건문을 사용해 재미있는 결과를 얻을 수 있습니다. 올림픽 참가국 전체 데이터나 이전 올림픽 데이터도 있다면 더 다양한 결과를 얻을 수 있답니다. 예를 들어, 각 나라의 메달 성적 추이가 어떻게 되는지도 알 수 있겠죠. 그러면 올림픽에 참여할수록 메달 성적이 좋아지는지도 알 수 있습니다. 경제 성장 데이터가 있다면 '올림픽 개최 이후 경제가 성장하는가?'도 직접 확인해 볼 수 있고요. 또는 '경제 규모와 올림픽 순위는 비례하는가?'에 대한 답도 직접 분석할 수 있습니다.

6장에서 배운 리스트와 제어구조를 이용한 리스트 다루기는 뒤에서 진행할 데이터 분석의 바탕이 됩니다. 지금까지 과정을 잘 이해했다면 데이터 분석 프로젝트에서 잘 활용할 수 있습니다.

1분 퀴즈

4 다음 그림은 모의고사의 목표 점수와 실제 점수를 나타낸 표입니다.

월	목표 점수	실제 점수
3월	82	83
4월	85	69
6월	75	88
9월	92	89
10월	100	97

이 표를 이차원 리스트로 만들면 다음 코드와 같습니다. 다음 설명 중 틀린 것을 고르세요.

```
exam_score = [['3월', 82, 83],
              ['4월', 85, 69],
              ['6월', 75, 88],
              ['9월', 92, 89],
              ['10월', 100, 97]]
for row in exam_score:
    if (가):
        (나)
```

① 6월의 목표 점수와 실제 점수를 출력하려면 (가)에 `row[0] == '6월'`, (나)에 `print (row[1:])`이 들어가야 한다.

② 목표 점수가 90점 이상인 월을 출력하려면 (가)에 `row[1] >= 90`, (나)에 `print (row[0])`이 들어가야 한다.

③ 실제 점수가 85점 이상인 월을 출력하려면 (가)에 `row[-1] >= 85`, (나)에 `print (row[0])`이 들어가야 한다.

④ 목표 점수보다 더 높은 점수를 받은 월을 출력하려면 (가)에 `row[-1] > row[1]`, (나)에 `print(row[0])`이 들어가야 한다.

⑤ 목표 점수보다 더 낮은 점수를 받았을 때, 목표 점수와 실제 점수의 차이를 출력하려면 (가)에 `row[-1] > row[1]`, (나)에 `print(row[-1] - row[1])`이 들어가야 한다.

정답 및 해설

6

마무리

1 리스트

여러 요소를 묶어 표현한 데이터 타입으로, 각 요소가 숫자 인덱스와 대응된다. 인덱스는 0부터 시작하는 점에 유의한다.

형식

```
리스트명 = [요소1, 요소2 …]
```

2 리스트에 사용할 수 있는 명령어

구분	형식
리스트 길이 구하기	`len(리스트명)`
리스트에 데이터 추가하기	`리스트명.append(데이터)`
리스트의 데이터 삭제하기	`리스트명.remove(데이터)` `del 리스트명[인덱스]`

리스트 슬라이싱하기	`리스트명[시작 인덱스:종료 인덱스]`
	(시작 인덱스부터 종료 인덱스 - 1까지 슬라이싱)
	`리스트명[:종료 인덱스]`
	(처음부터 종료 인덱스 - 1까지 슬라이싱)
	`리스트명[시작 인덱스:]`
	(시작 인덱스부터 끝까지 슬라이싱)

리스트 정렬하기	`리스트명.sort()` (`reverse=True` 옵션을 사용하면 내림차순 정렬, 옵션을 사용하지 않으면 오름차순 정렬이 기본값)

3 문자열 슬라이싱하기

구분	형식

인덱스를 사용한 문자열 슬라이싱	`변수[시작 인덱스:종료 인덱스]`
	`변수[:종료 인덱스]`
	(처음부터 종료 인덱스 - 1까지 슬라이싱)
	`변수[시작 인덱스:]`
	(시작 인덱스부터 끝까지 슬라이싱)

`split()`을 사용한 문자열 나누기	`변수.split(기준 문자)` (기준 문자를 설정하지 않으면 공백을 기준으로 자동 분리)

4 이차원 리스트

리스트 안에 리스트가 있는 리스트다. 행 인덱스와 열 인덱스가 있으며, 행 인덱스를 먼저 쓰고 열 인덱스를 뒤에 쓴다.

Self Check

1 다음 그림처럼 2개의 리스트로 도서 제목과 도서 가격을 관리하려고 합니다. 도서 제목을 저장하는 리스트는 `book_name`이고, 도서 가격을 저장하는 리스트는 `book_price`입니다. `book_name[0]`에 저장된 도서의 가격은 `book_price[0]`에, `book_name[1]` 도서의 가격은 `book_price[1]`에 저장하는 방식으로 도서 제목과 가격을 저장한다고 합시다.



Copyright © Gilbut, Inc. All rights reserved.



Copyright © Gilbut, Inc. All rights reserved.

정리한 내용을 이차원 리스트로 만들면 다음과 같습니다. 이 코드를 바탕으로 물음에 답하세요.

```
import random
book_name = ['역사탐험대', '파이썬', '학습법', '영단어', '여행에세이', '삼국지']
book_price = []
```

1) `book_name`이 코드와 같을 때 15,000~25,000원 사이의 무작위 정수를 `book_price`에 도서 가격으로 추가하는 코드를 이어서 작성하세요. 그리고 `book_name`과 `book_price`를 출력하세요.

2) 가장 비싼 가격의 도서 제목을 출력해 보세요.

2 다음 그림은 오디션 프로그램 참가자들의 순위표입니다.

순위	참가자	이번 주 득표수	지난주 득표수
1	라이언	956412	564553
2	니니즈	861832	927175
3	어피치	796354	593175
4	프로도	534840	864326
5	네오	387896	648367

이 표를 이차원 리스트로 만들면 다음과 같습니다. 이 코드를 바탕으로 물음에 답하세요.

```
ranking = [[1, '라이언', 956412, 564553],
           [2, '니니즈', 861832, 927175],
           [3, '어피치', 796354, 593175],
           [4, '프로도', 534840, 864326],
           [5, '네오', 387896, 648367]]
```

1) 참가자의 순위-이름을 쌍으로 한 번에 출력하는 코드를 작성하세요.

2) 이번 주 득표수가 50만 표가 넘는 참가자의 이름과 이번 주 득표수를 같이 출력하세요.

3) 지난주보다 득표수가 떨어진 참가자가 누구인지 득표수가 얼마나 떨어졌는지를 출력하세요.

4) 지난주와 이번 주 득표수 중 한 번이라도 80만 표를 넘은 참가자를 출력하세요.

정답 및 해설